

Trees, DFS traversal.





Review - Lecture 2

- Definitions: Path, cycle, simple path, simple cycle

- Euler path: a path of length m that traverses every edge in E exactly once. If it begins and ends at the same node, then the path is a cycle.

Review - Lecture 2

- Definitions: Path, cycle, simple path, simple cycle

- Euler path: a path of length m that traverses every edge in E exactly once. If it begins and ends at the same node, then the path is a cycle.

- A connected graph has an Euler cycle if and only if all its vertices have an even degree.
- A connected graph has an Euler path which is not a cycle if and only if exactly two of its vertices have an odd degree.

- Algorithm for finding an Euler cycle.

Review - Lecture 2

Conditions for directed graphs:

1. The underlying undirected graph is connected.
2. Either (cycle) $d_{in}(v)=d_{out}(v)$ for all v, or

(path)

- $d_{in}(v)=d_{out}(v)$ for all but two vertices,
- for one vertex *a,* $d_{out}(a) = d_{in}(a) + 1$ (a will be first)
- for one vertex *b,* $d_{in}(b) = d_{out}(b) + 1$ (b will be last).

Review - Lecture 2

De-Bruijn sequence: a (circular) sequence $a_0 a_1 \ldots a_{L-1}$ over Σ such that for every word w of length n over Σ there exists a unique i such that $a_i a_{i+1} \ldots a_{i+n-1} = w$.

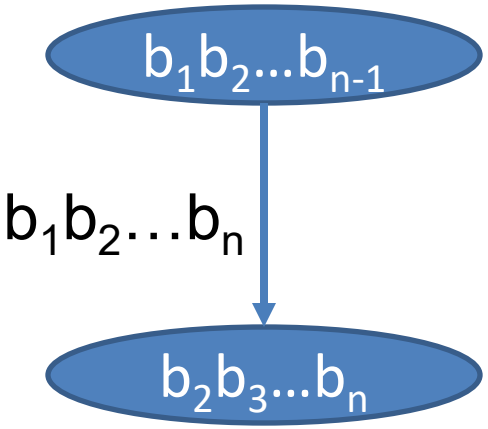
Theorem: For every positive integers σ and n there exists a de Bruijn sequence.

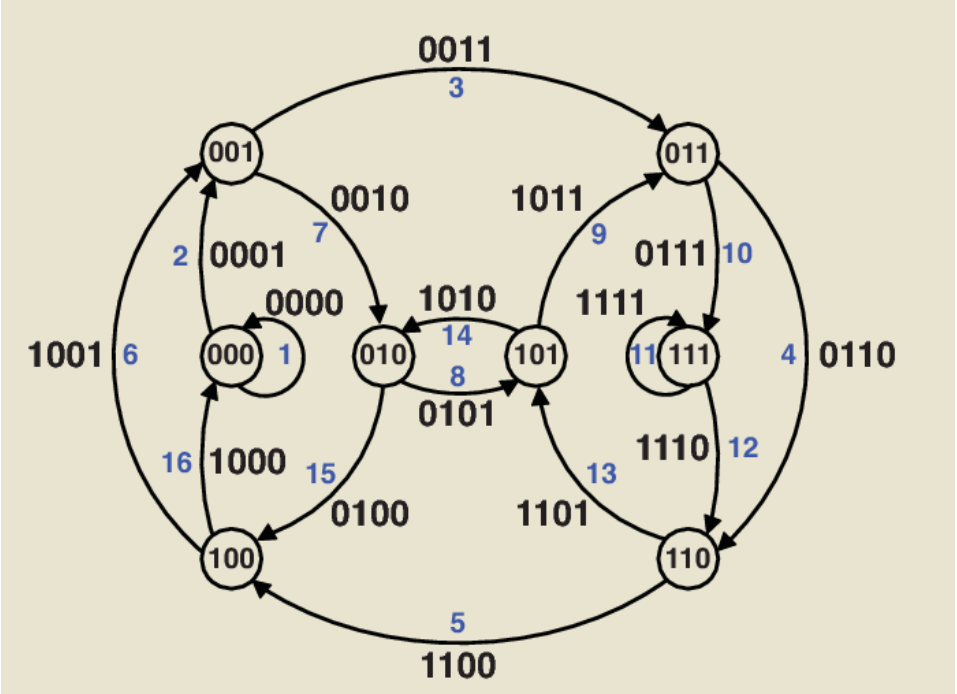
Proof: By construction.

Review - Lecture 2

Overview: Given n, σ, we construct a directed graph, such that every word of length n over σ, corresponds to an edge in G.

An Euler cycle ≡ a De-Bruijn sequence.





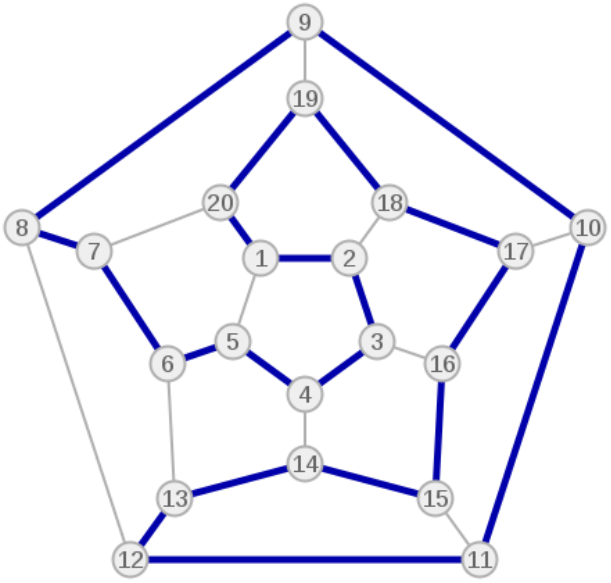
Example: n=4, σ=2

Review - Lecture 2

Hamilton path (cycle) visits every node exactly once.
= A simple path (or cycle) of length n-1 (or n)

- No efficient (poly-time) algorithm is known.

However, there are some known sufficient conditions.

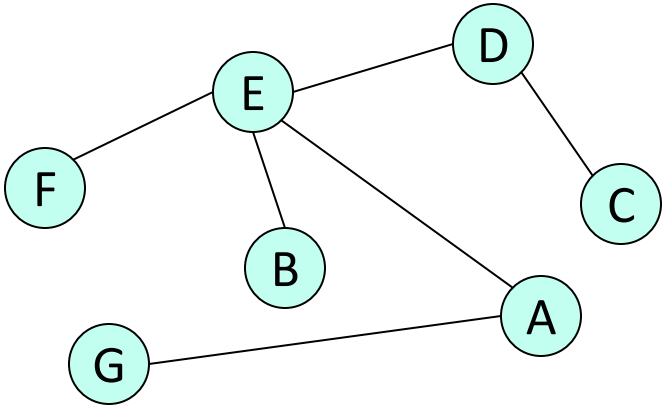


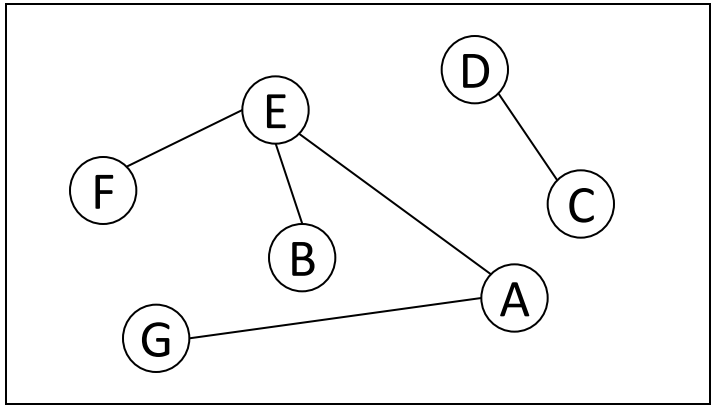


Trees

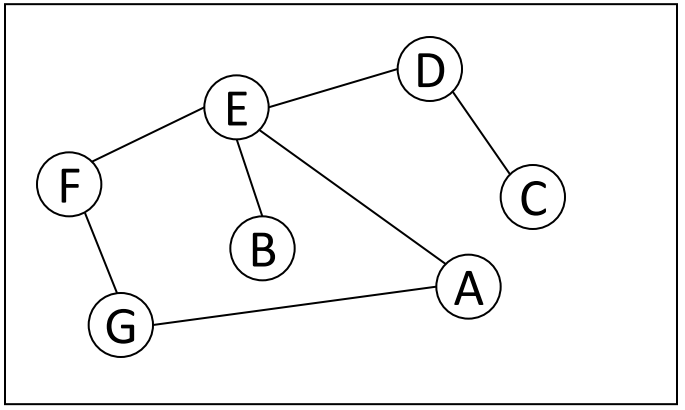


- An undirected graph is a tree if it is connected and contains no cycles.





Not trees



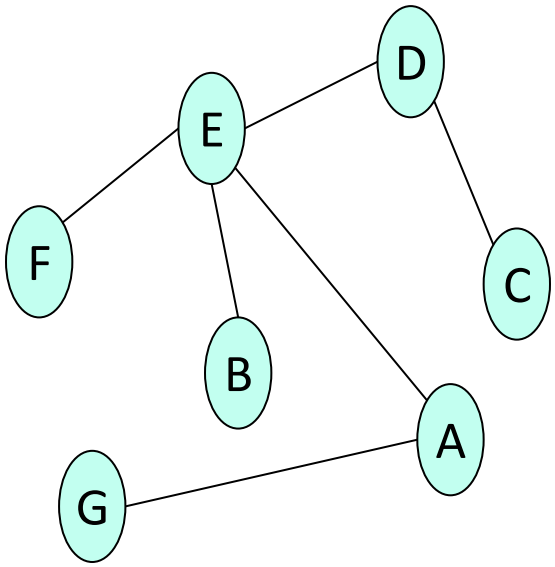
Alternative Definitions of Undirected Trees

a. G is a tree (connected and contains no cycles).

b. G is cycles-free, but if any new edge is added to *G,* a cycle is formed.

c. For every two vertices there is a unique simple path connecting them.

d. G is connected, but if any edge is deleted from *G,* the connectivity of G is interrupted.



Alternative Definitions of Undirected Trees

Theorem: All four definitions are equivalent.

Proof: In class.

Sketch: prove four claims:

1. (a) implies (b)
2. (b) implies (c)
3. (c) implies (d)
4. (d) implies (a)



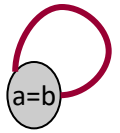
Some more equivalent definitions:

1. G is connected and has n-1 edges.
2. G is acyclic and has n-1 edges.

Proof: Alternative tree characterization.

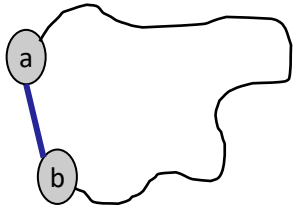
(a) G is a tree (connected and cycles-free).
(b) G is cycles-free, but if any new edge is added to G, a cycle is formed.

(a) $\Rightarrow$ (b):



Let e=(a,b) be a new edge, that is, e $\notin$ E, a,b$\in$V.

If a = b, then e forms a self-loop cycle.



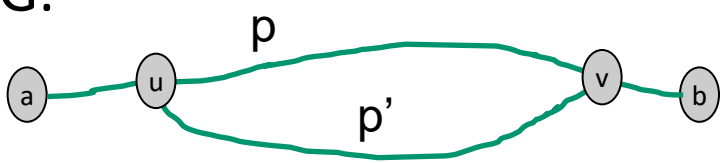
If a $\neq$ b, there is a path in G (without e) connecting a and b; if we add e, this path with e forms a cycle.

(b) G is cycles-free, but if any new edge is added to G, a cycle is formed.
(c) For every two vertices there is a unique simple path connecting them.

(b) $\Rightarrow$ (c): Let a and b be any two vertices of G.

- Exists: If there is no path between a and b, then we can add an edge (a,b) without creating a cycle, a contradiction.
- Simple: If a path is not simple, a cycle exists.
- Unique: Assume by contradiction that there are at least two paths, p and p', between a and b.

We show that G includes a cycle.
Let u be the first vertex in which the paths separate (possibly u=a), let v be the first vertex in which the paths meet again (possibly v=b). The two disjoint subpaths between u and v form a cycle in G.

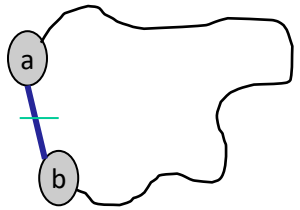


(c) For every two vertices there is a unique simple path connecting them.
(d) G is connected, but if any edge is deleted, the connectivity of G is interrupted.

(c) $\Rightarrow$ (d):

It is given that G is connected.

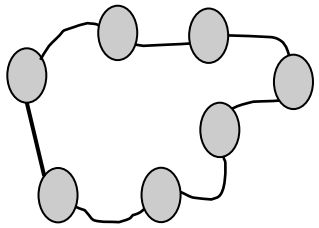
Assume that an edge e=(a,b) is deleted. If there is now (after the deletion of e) a path between a and b, then G has more than one simple path between a and b.



(d) G is connected, but if any edge is deleted, the connectivity of G is interrupted.
(a) G is a tree (connected and cycles-free).

(d) $\Rightarrow$ (a):

We assume that G is connected and that no edge can be deleted without interrupting the connectivity.
If G contains a simple cycle, any edge on this cycle can be deleted without interrupting the connectivity.
Thus, G is cycles-free.



משפט אפיון לעצים לא מכוונים

משפט: ארבעת התנאים הבאים שקולים:

א- G הוא עץ (קשיר וחסר מעגלים)

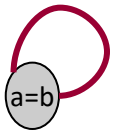
ב- G חסר מעגלים וכל הוספת קשת יוצרת מעגל

ג- בין כל שני צמתים יש מסלול פשוט יחיד

ד- G קשיר וכל הורדת קשת פוגעת בקשירות.

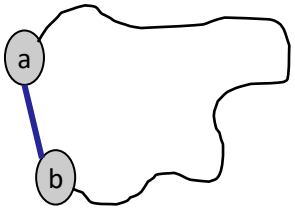
הוכחה: נראה א $\Leftarrow$ב $\Leftarrow$ ג $\Leftarrow$ ד $\Leftarrow$ א

א $\Leftarrow$ב : G קשיר וחסר מעגלים.



תהי e=(a,b) קשת חדשה שאינה ב-G, כלומר $e\notin E$, $a,b\in V$.

אם a=b, נוצר מעגל (חוג עצמי).



אם $a\neq b$ אז מכיוון ש- G קשיר קיים איזשהו מסלול בין a ל-b, ויחד עם הקשת נסגר מעגל.

א- G הוא עץ (קשיר וחסר מעגלים)

ב- G חסר מעגלים וכל הוספת קשת יוצרת מעגל

ג- בין כל שני צמתים יש מסלול פשוט יחיד

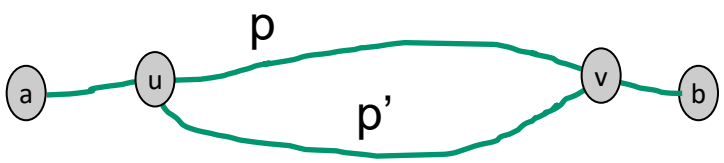
ד- G קשיר וכל הורדת קשת פוגעת בקשירות.

ב $\Leftarrow$ ג : יהיו a,b שני צמתים ב-G.

- אם אין מסלול ביניהם אז הוספת הקשת (a,b) לא סוגרת מעגל בסתירה לנתון. לכן בהכרח קיים לפחות מסלול אחד בינהם.
- כל מסלול בין a ל-b הוא פשוט, אחרת יש בגרף מעגל.
- נראה שיחיד.

נניח בשלילה שיש יותר ממסלול אחד בין a ל-b.

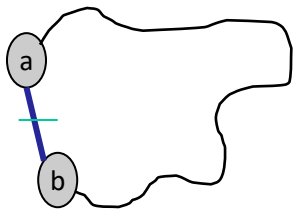
יהיו p,p' שני מסלולים שונים בין a ל-b. יהי u הצומת הראשון שבו המסלולים מתפצלים (יתכן u=a) ויהי v הצומת הראשון שבו p ו-p' שוב נפגשים (יתכן v=b). שני קטעי המסלולים מהפיצול ועד המפגש מהווים מעגל בסתירה לנתון.



א- G הוא עץ (קשיר וחסר מעגלים)
ב- G חסר מעגלים וכל הוספת קשת יוצרת מעגל
ג- בין כל שני צמתים יש מסלול פשוט יחיד
ד- G קשיר וכל הורדת קשת פוגעת בקשירות.

ג ⇐ ד :

תהי e=(a,b) קשת בגרף.
מהנתון קיים מסלול פשוט יחיד בין a ל-b. אם אחרי הורדת e עדיין יש מסלול ביניהם, סימן שלפני ההורדה היה יותר ממסלול אחד. סתירה.



א- **G הוא עץ (קשיר וחסר מעגלים)**

ב- G חסר מעגלים וכל הוספת קשת יוצרת מעגל

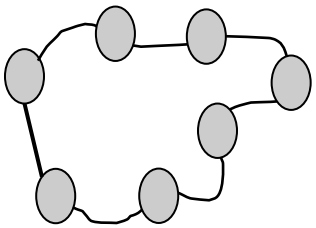
ג- בין כל שני צמתים יש מסלול פשוט יחיד

ד- G קשיר וכל הורדת קשת פוגעת בקשירות.

ד ⇐ א :

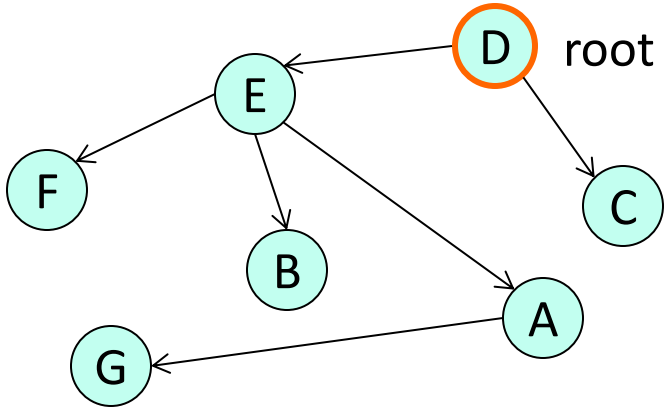
G קשיר וכל הורדת קשת פוגעת בקשירות.

אם G כולל מעגל אז אפשר להוריד כל אחת מקשתות המעגל מבלי לפגוע בקשירות, לכן G חסר מעגלים.



Directed Trees

- A directed graph is a directed tree if it has a root and its underlying undirected graph is a tree.
- $r \in V$ is a root if every vertex $v \in V$ is reachable from r; i.e., there is a directed path which starts in r and ends in v.

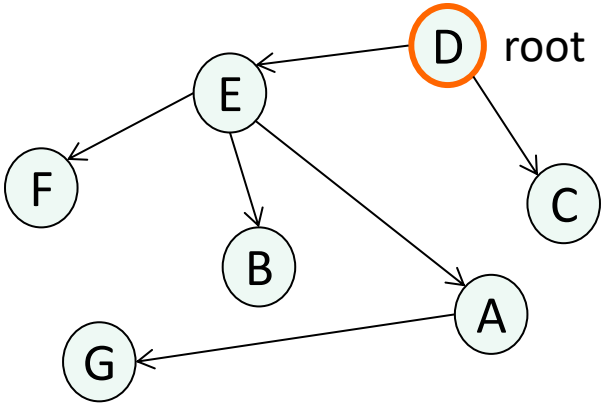


Alternative Definitions of Directed Trees

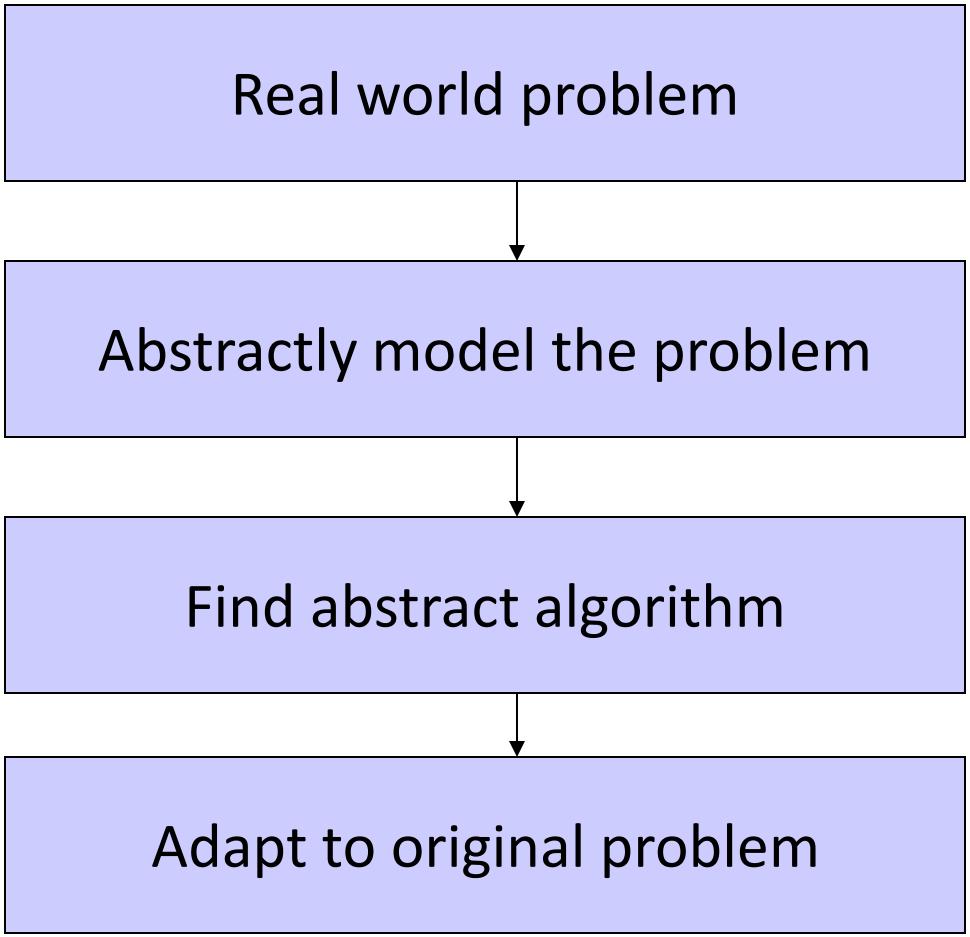
(a) G is a directed tree.

(b) G has a root from which there is a unique directed path to every vertex.

(c) G has a root *r* for which $d_{in}(r) = 0$ and for every other vertex *v*, $d_{in}(v) = 1$.

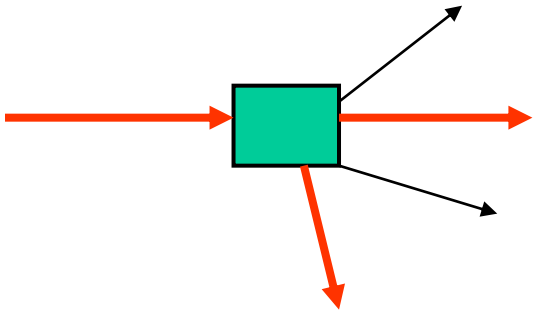


Applied Algorithm Scenario



Broadcasting in a Network

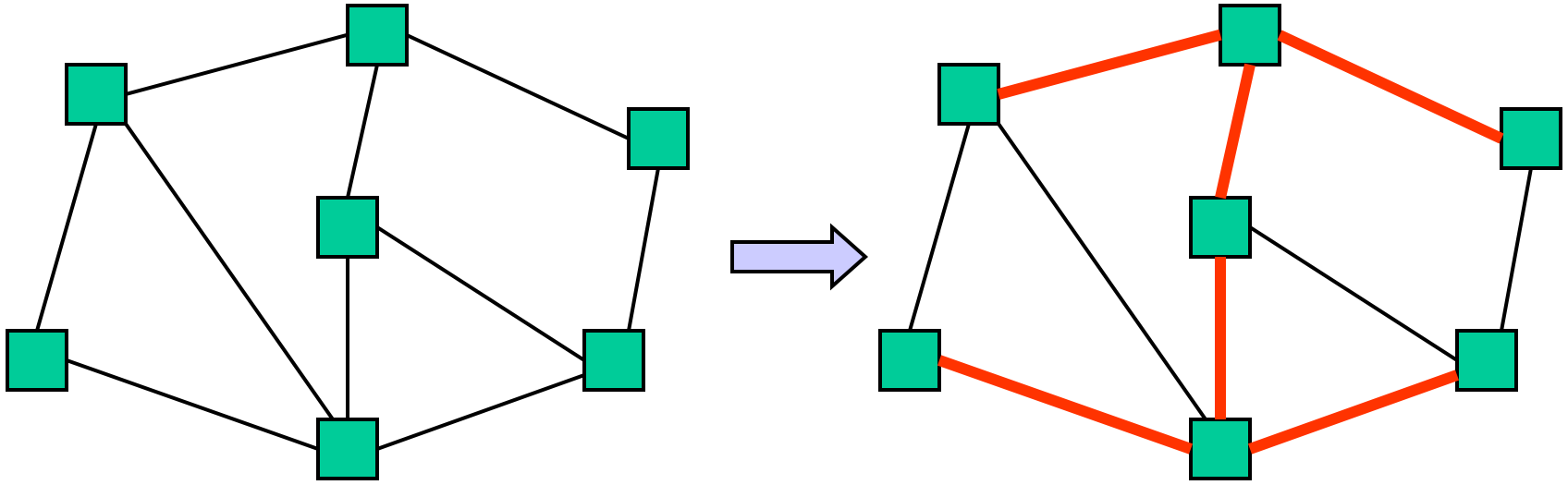
- Network of Routers
  - Goal: organize the routers to efficiently broadcast messages to each other.



- A router that gets a message, duplicate and send it to some of its neighbors.
- Eventually all routers get the message

Goal: Minimize the number of messages.

Spanning Tree in a Graph

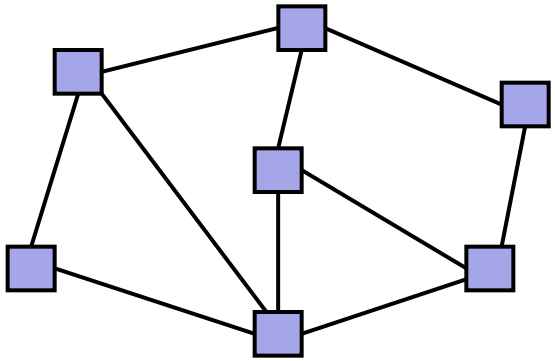


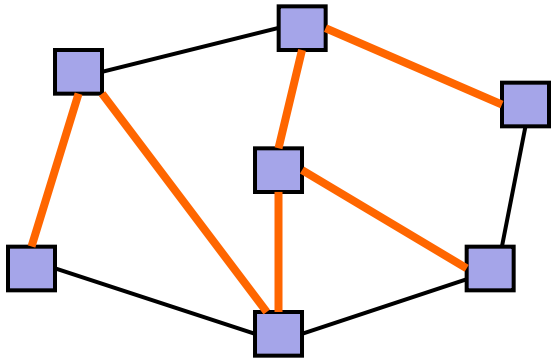
Vertex = router
Edge = link between routers

Spanning tree
- Connects all the vertices
- No cycles

Spanning Tree Problem

- Input: An undirected graph G = (V,E). G is connected.
- Output: E’ ⊆ E such that
  - (V, E’) is a connected graph
  - (V, E’) has no cycles



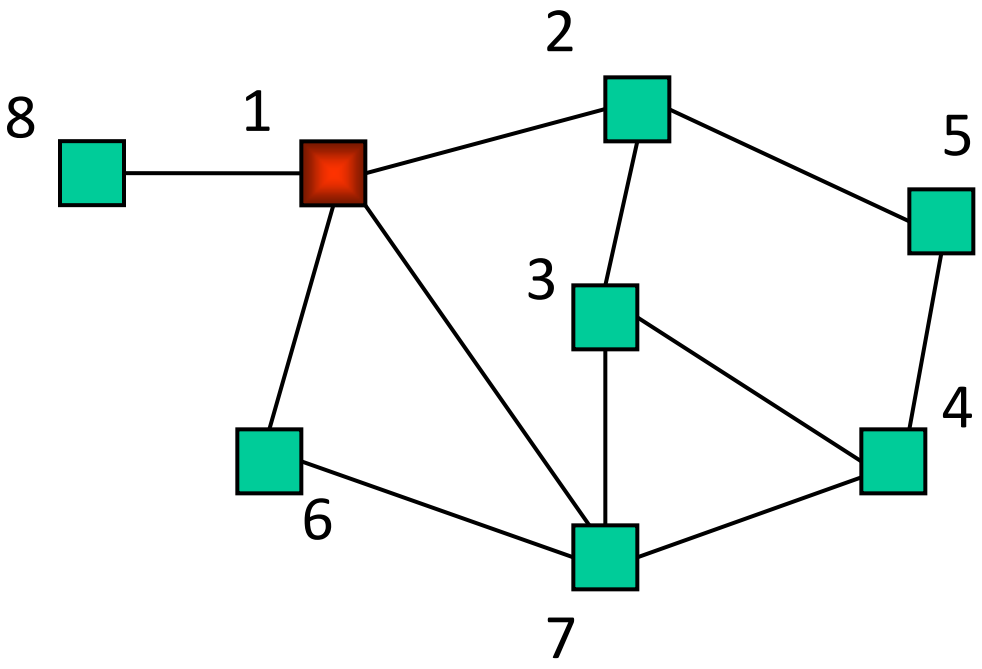


Depth First Search Algorithm

- Recursive marking algorithm
- Initially every vertex is unmarked

```
DFS(i: vertex)
   mark i;
   for each j adjacent to i do
      if j is unmarked then DFS(j)
end {DFS}
```

Example of Depth First Search

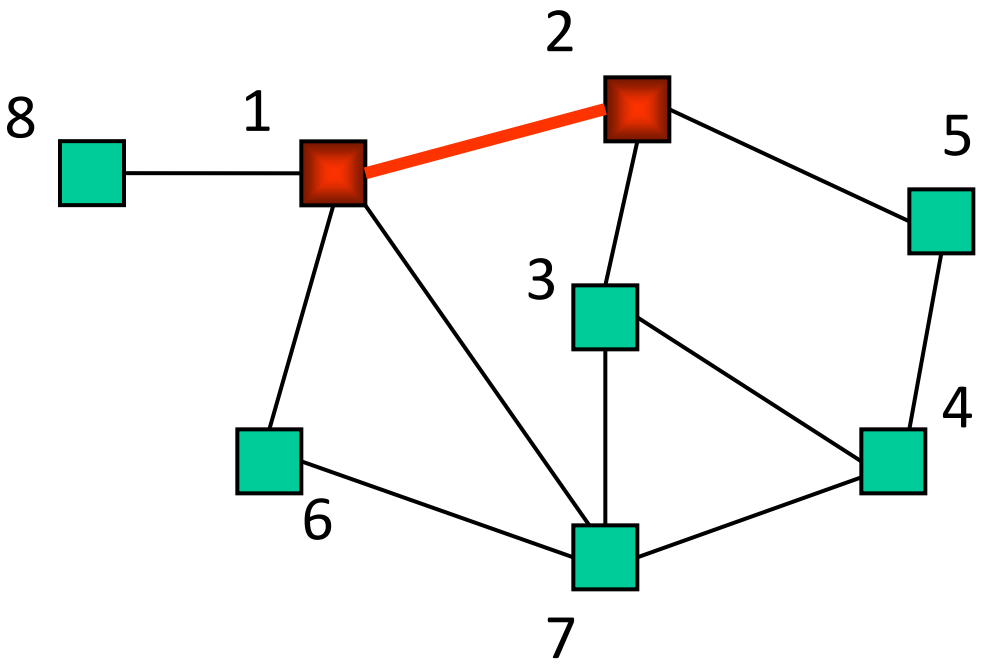


DFS(1)

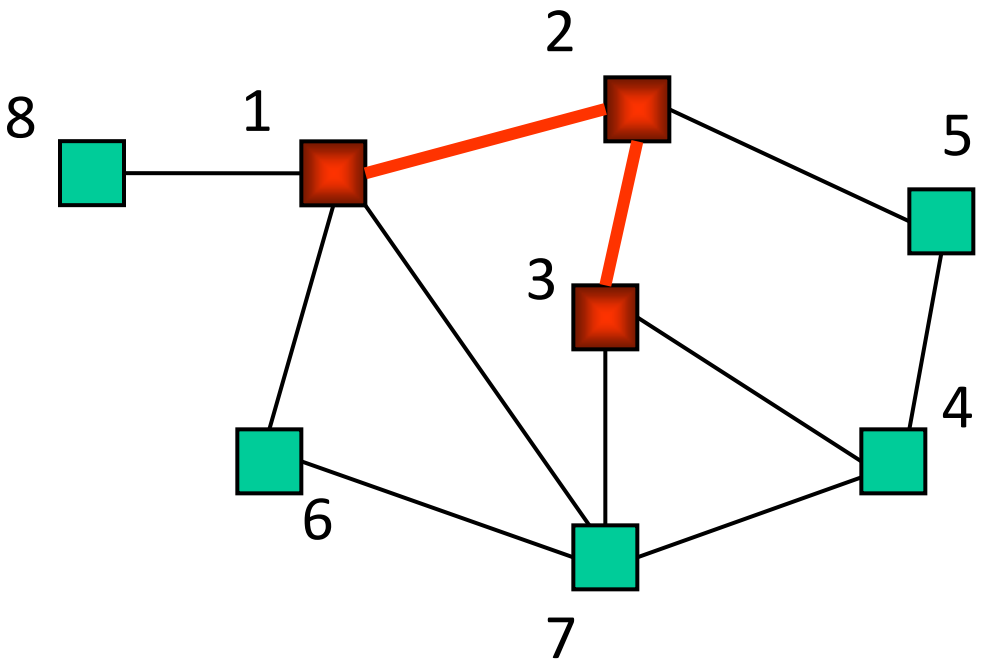
Running stack

Example Step 2

DFS(2)
DFS(1)

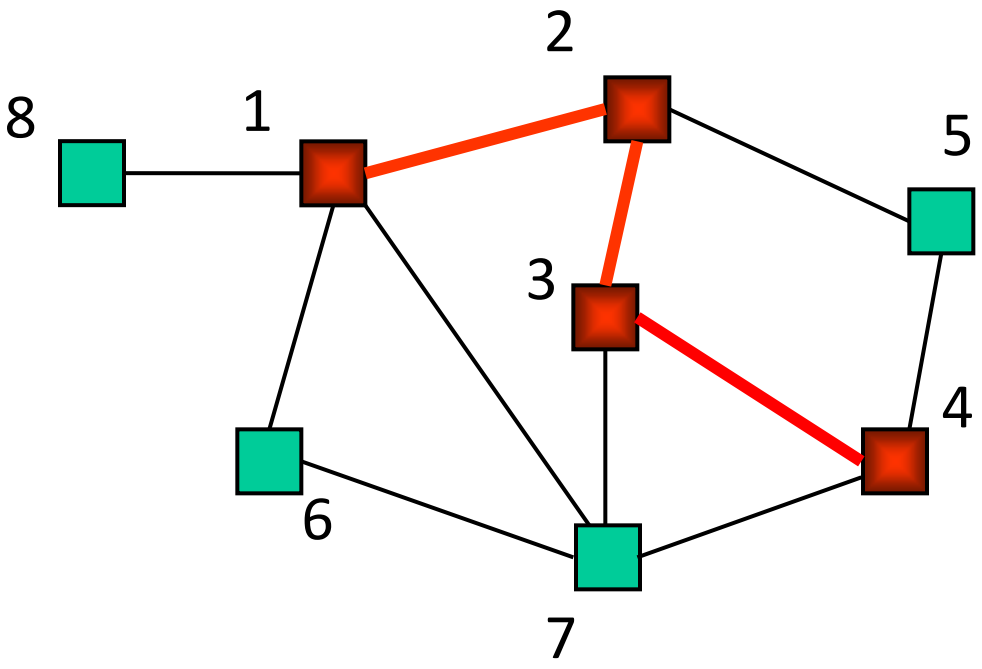


Example Step 3



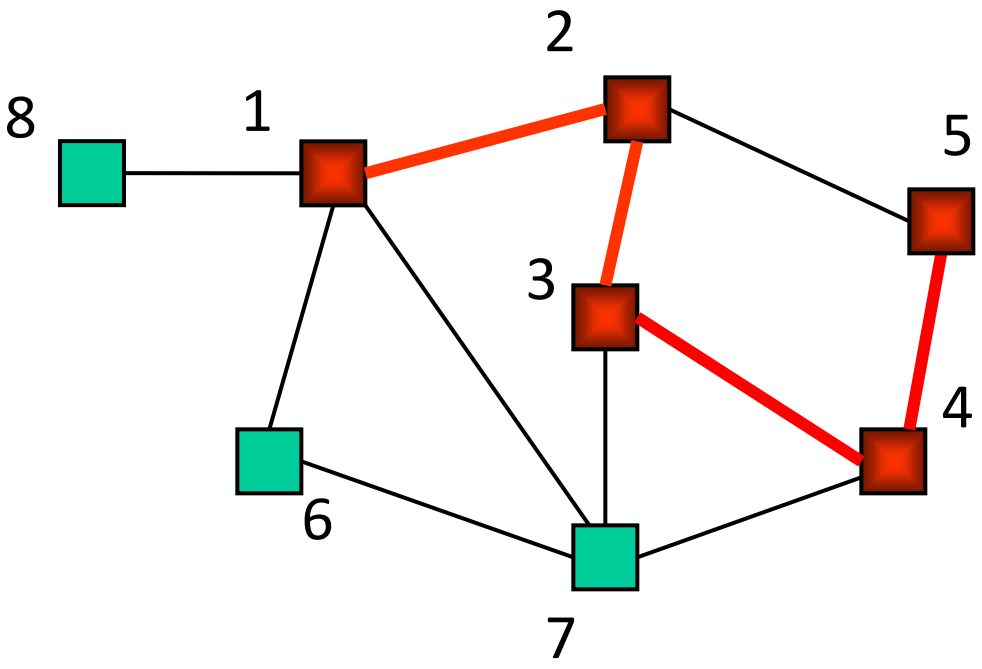
DFS(3)
DFS(2)
DFS(1)

Example Step 4



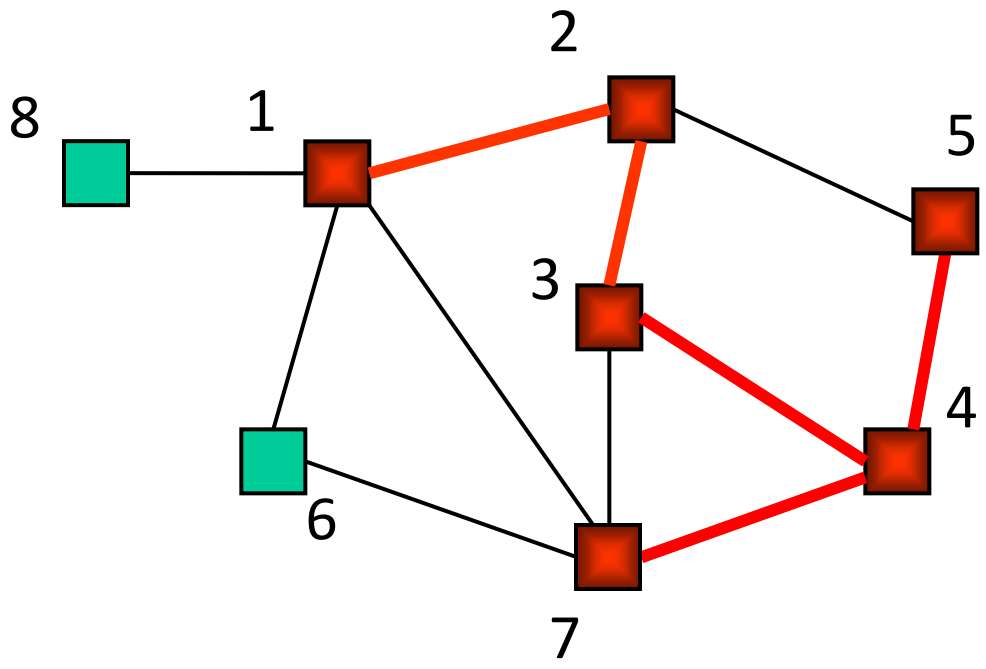
DFS(4)
DFS(3)
DFS(2)
DFS(1)

Example Step 5



DFS(5)
DFS(4)
DFS(3)
DFS(2)
DFS(1)

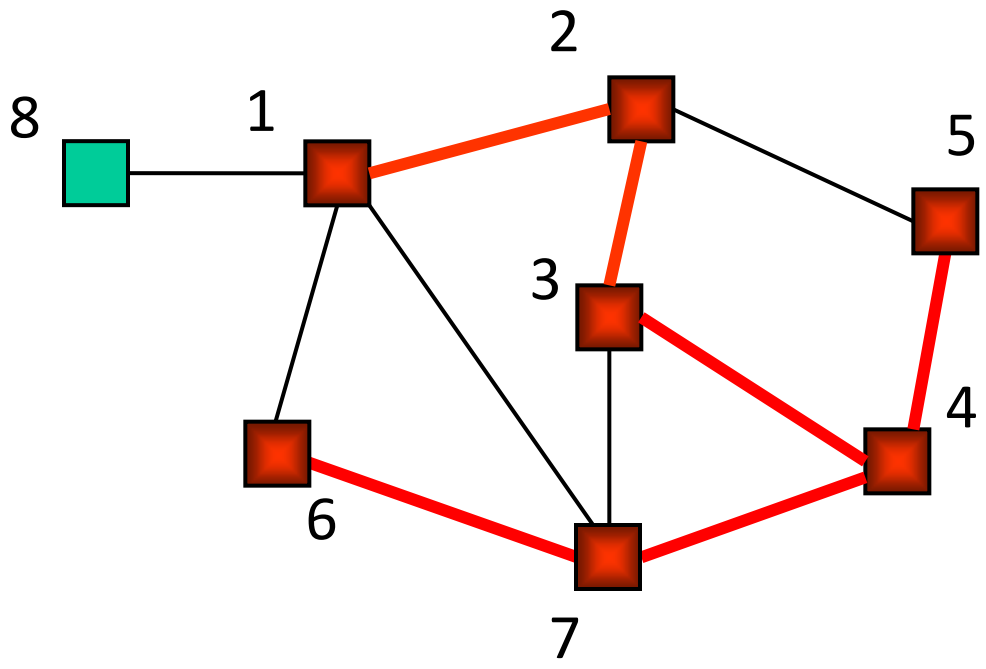
Example Step 6



DFS(7)
DFS(4)
DFS(3)
DFS(2)
DFS(1)

Note: 5 is not in the stack anymore. It is the first vertex to complete its DFS run.

Example Step 7

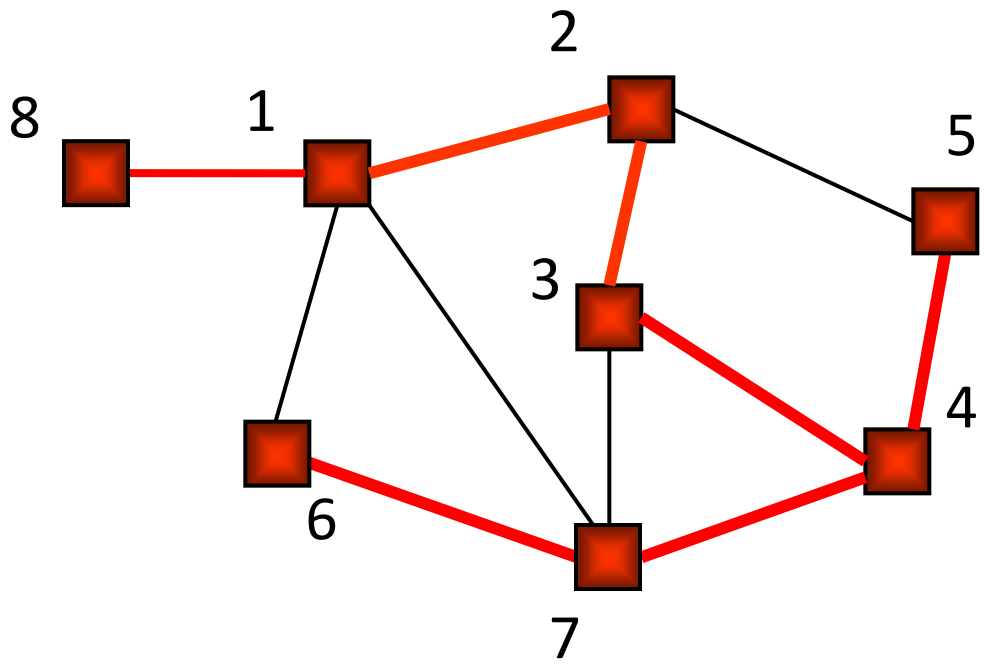


DFS(6)
DFS(7)
DFS(4)
DFS(3)
DFS(2)
DFS(1)

Start backtracking to 1

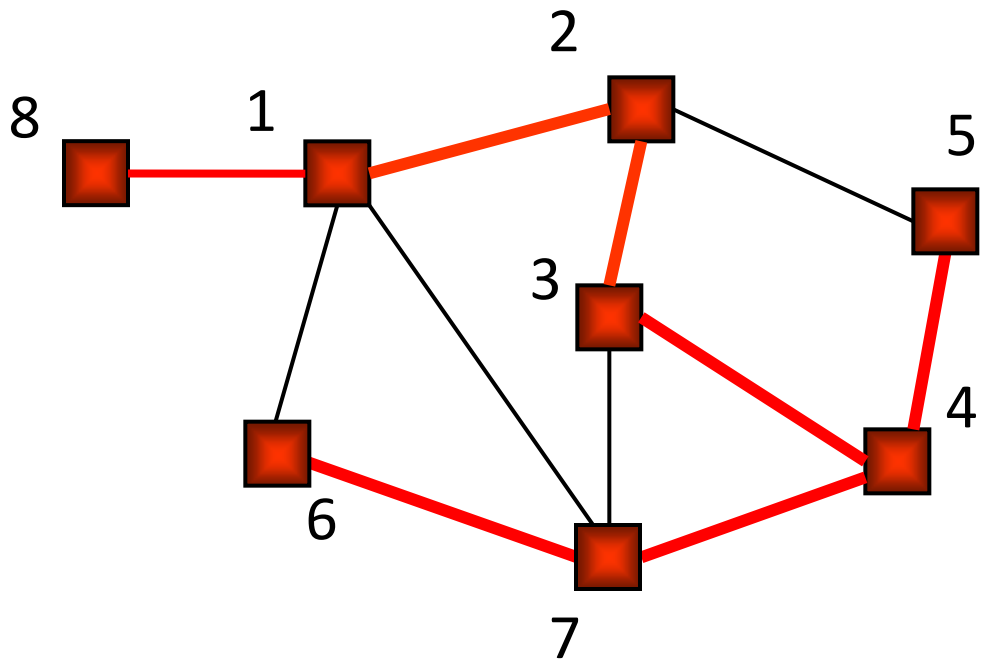
Example Step 8

DFS(8)
DFS(1)



All vertices are marked but DFS does not terminate yet, the stack is still not empty.

Example Step 9



The stack is empty, meaning that all vertices verified that all their neighbors are marked.

the edges traversed in the DFS form a spanning tree.

Spanning Tree Algorithm

```
Main
E' := empty set;
ST(s);
end{Main}
```

An arbitrary vertex

```
ST(i: vertex)
    mark i;
    for each j adjacent to i do
        if j is unmarked then
            Add (i,j) to E';
            ST(j);
end{ST}
```

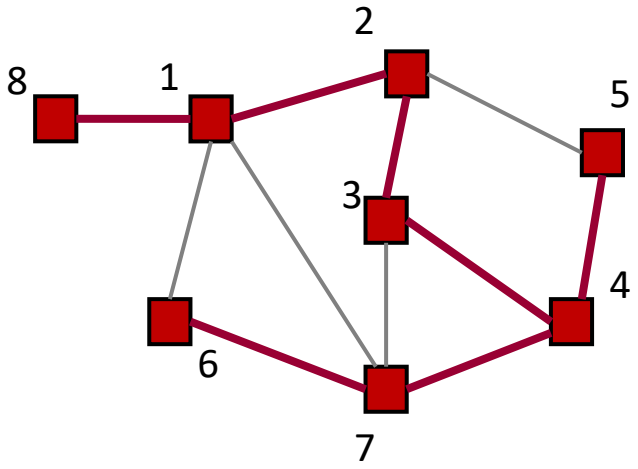
The addition to DFS

In DFS, whenever an unmarked node, j, is discovered by its neighbor i, add the edge (i,j) to the spanning tree.

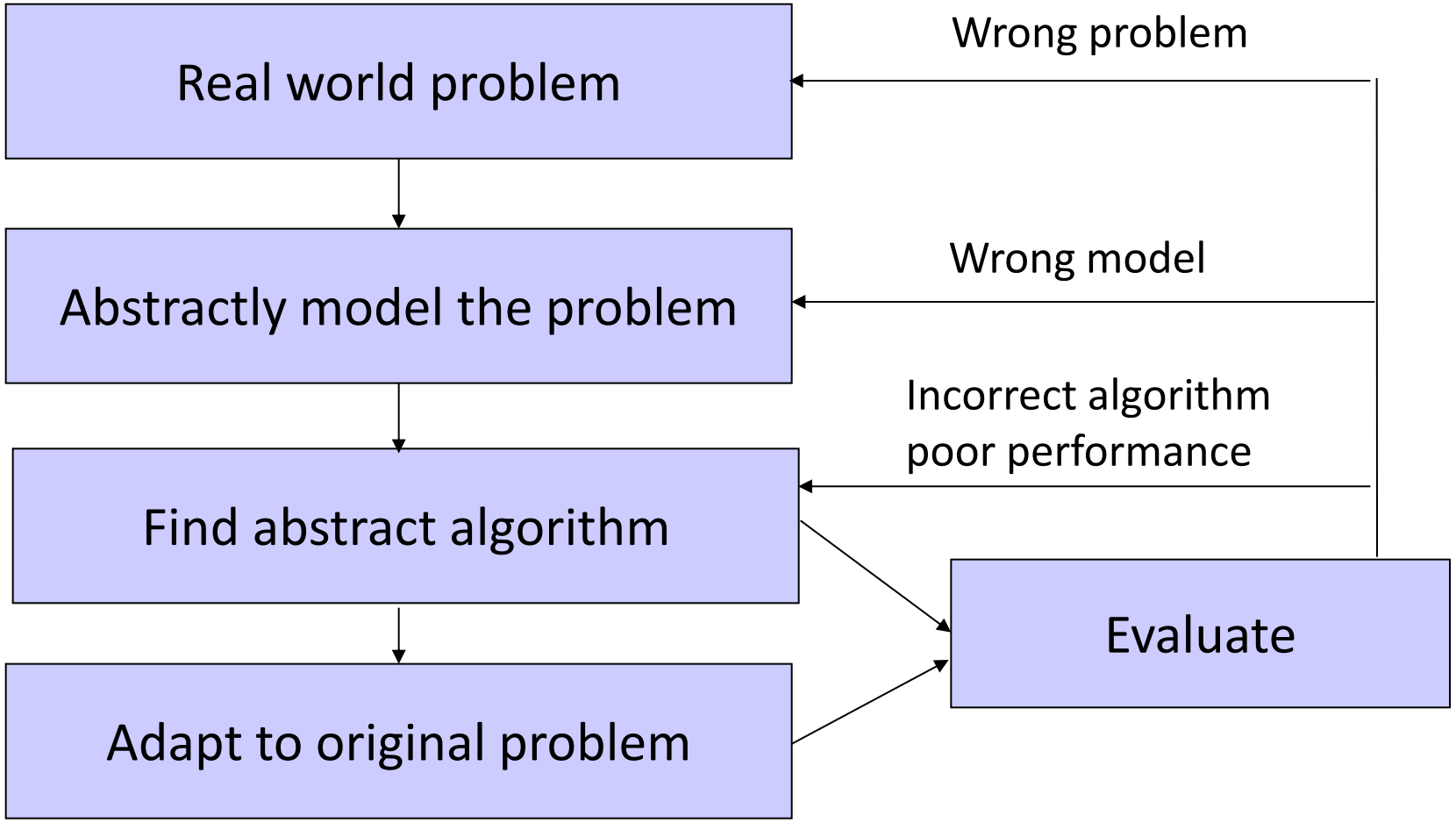
Edges Classification – DFS of undirected graph

Tree edges: belong to the spanning tree. Connect a node and the vertex that 'discovers' it.

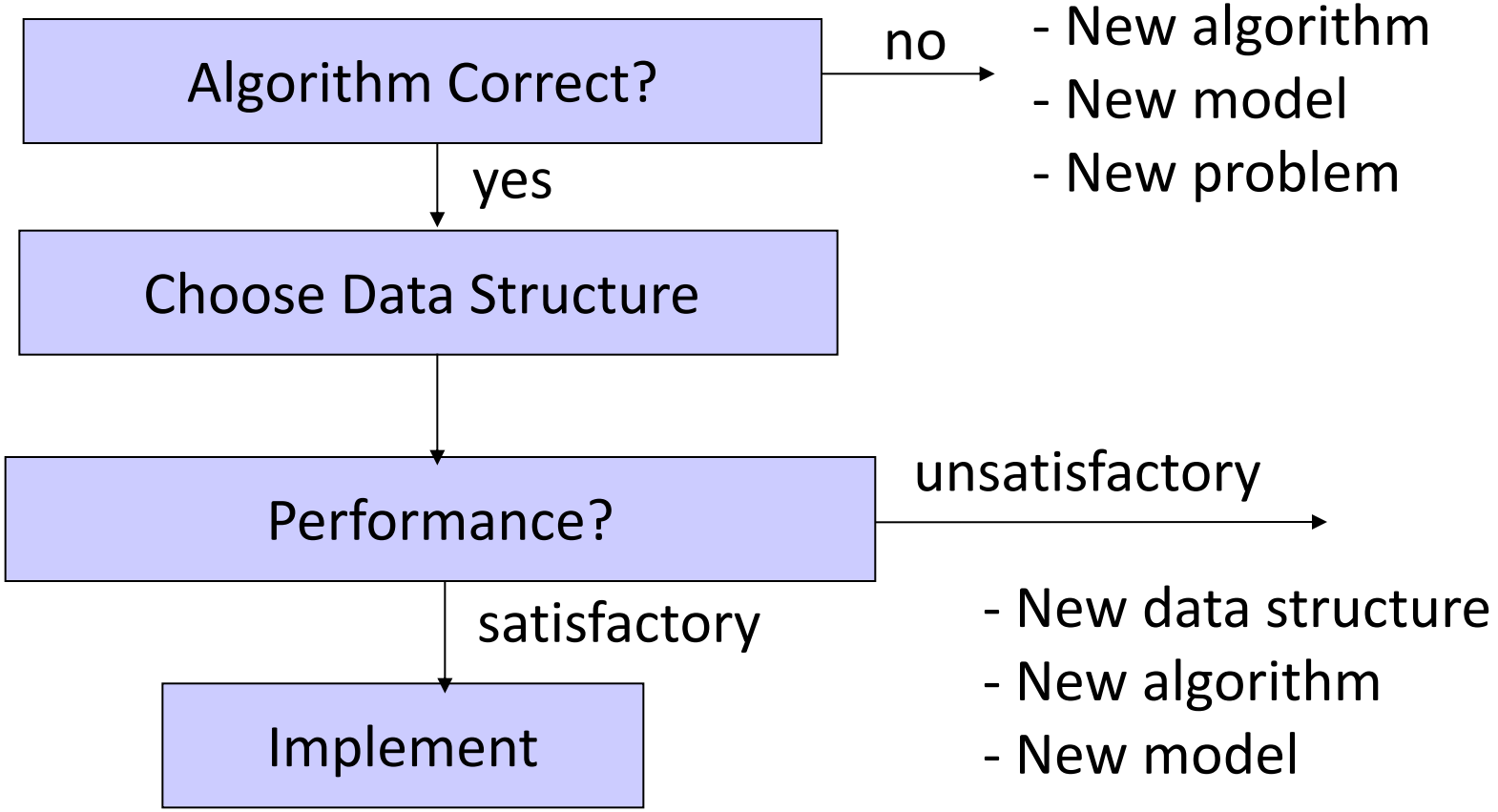
Back edges: all other edges. Connect a node and already-marked neighbor.



Applied Algorithm Scenario



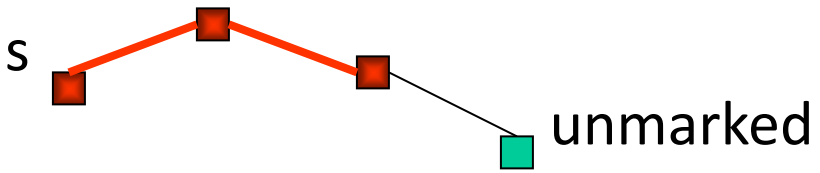
Evaluation Step Expanded



Correctness of ST Algorithm

Let T=(V,E') be the graph resulting from running DFS.

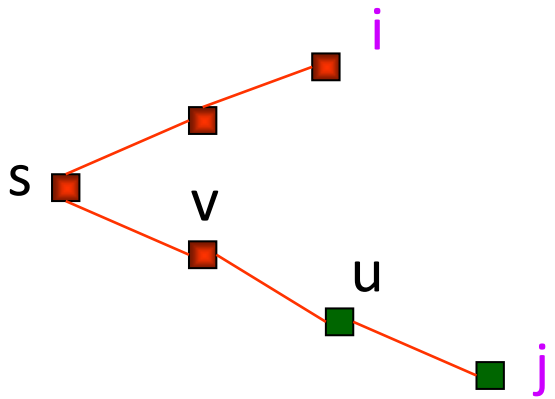
- Claim 1: There are no cycles in T.
- Proof: This is an invariant of the algorithm.
  - Each edge added to T goes from a vertex in T to a vertex not in T, therefore, no cycle is closed.



An invariant is a property of a program that is true before (and after) each iteration]

Correctness (cont.)

Claim 2: If G is connected then so is T

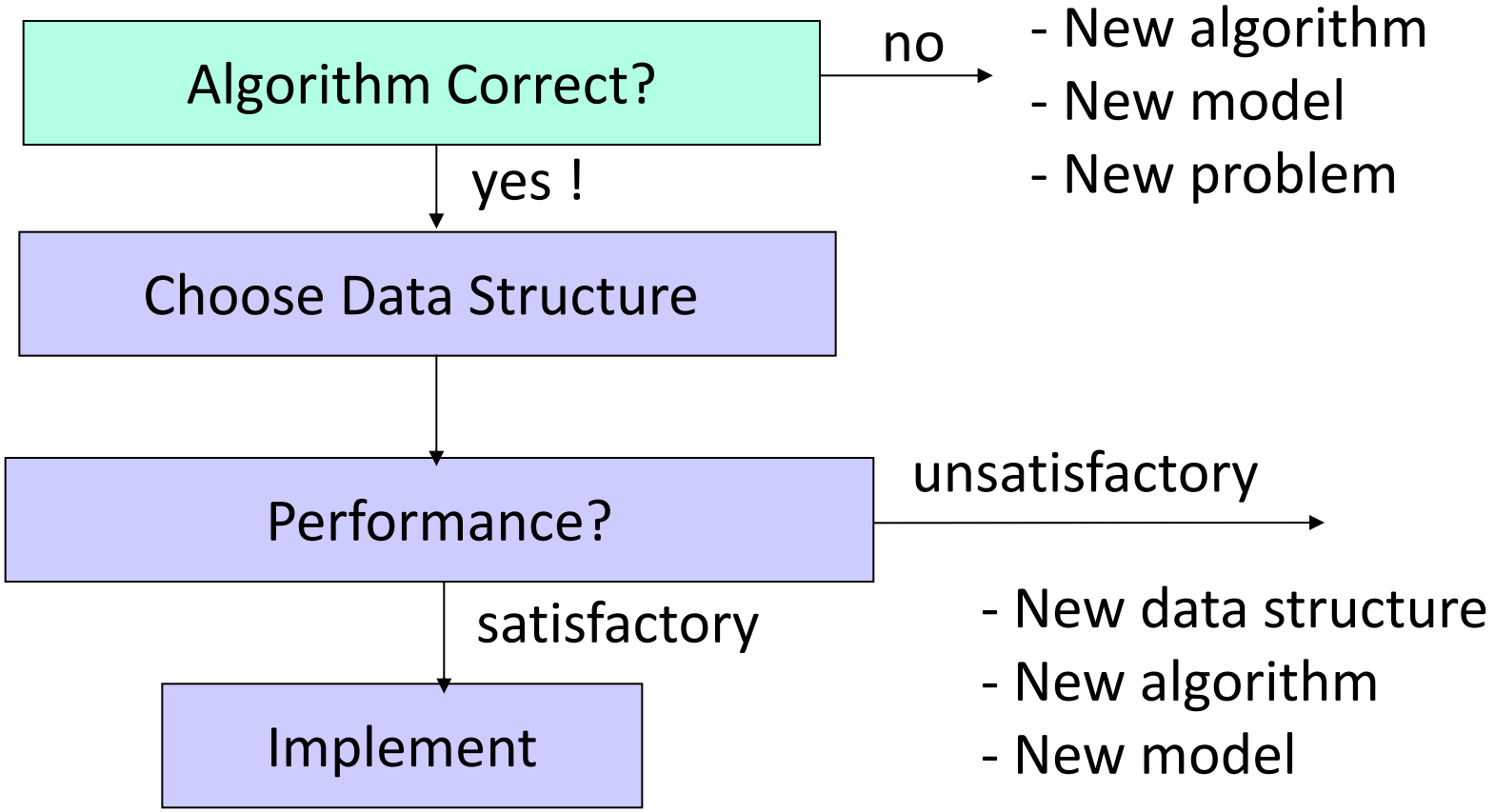


Proof: Consider two vertices i and j in V.

In G, there are paths connecting s and i and s and j. Assume by contradiction that i and j are not connected in T. It must be that at least one vertex along these paths is not marked.

Let u be a vertex closest to s which is not marked on these paths. Let v be u's neighbor in the path to s. The fact that u is unmarked is a contradiction to the DFS execution on v.

Data Structure Step



Data Structure Choice

- Both Adjacency matrix and Adjacency lists support depth first search
- As in many other algorithms, adjacency matrix is good for dense graphs, while adjacency lists are good for sparse graphs.

- Dense graph: $m = \Omega(n^2)$. For example "half of the nodes have degree n/5".
- Sparse graph: m=O(n). For example "5 nodes have degree n/2, and all other nodes have degree at most 8".

Most practical graphs are sparse.

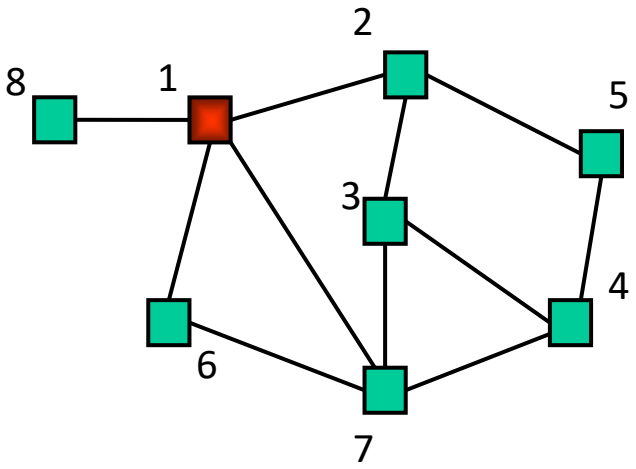
Spanning Tree with Adjacency Lists

```
Main
   G is array of adjacency lists;
   M[i] := 0 for all i;
   E' is empty;
   Spanning_Tree(s);
end{Main}
```

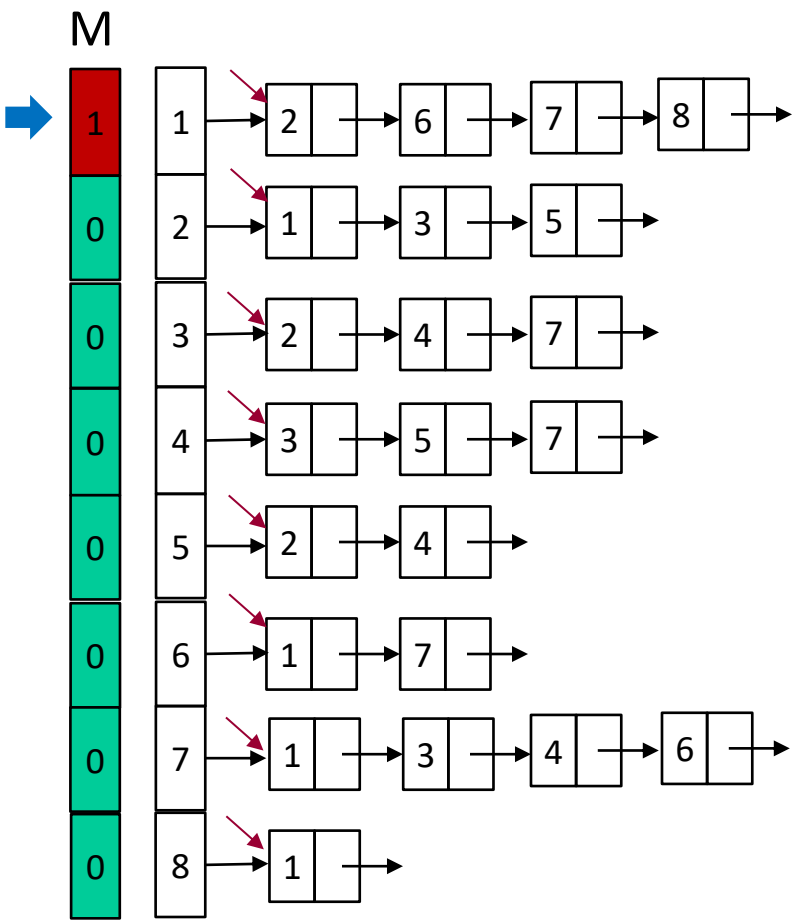
M is the marking array
(0/1 entry for each vertex).

```
ST(i: vertex)
   M[i] := 1;
   v := G[i];
   while (v ≠ null)
      j := v.vertex;
      if (M[j] = 0) then
         add {i,j} to E';
         ST(j);
      v := v.next;
end{ST}
```

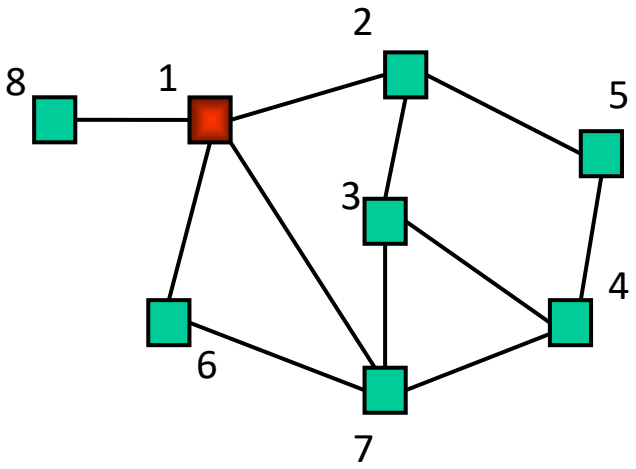
DFS Example – with adj. lists



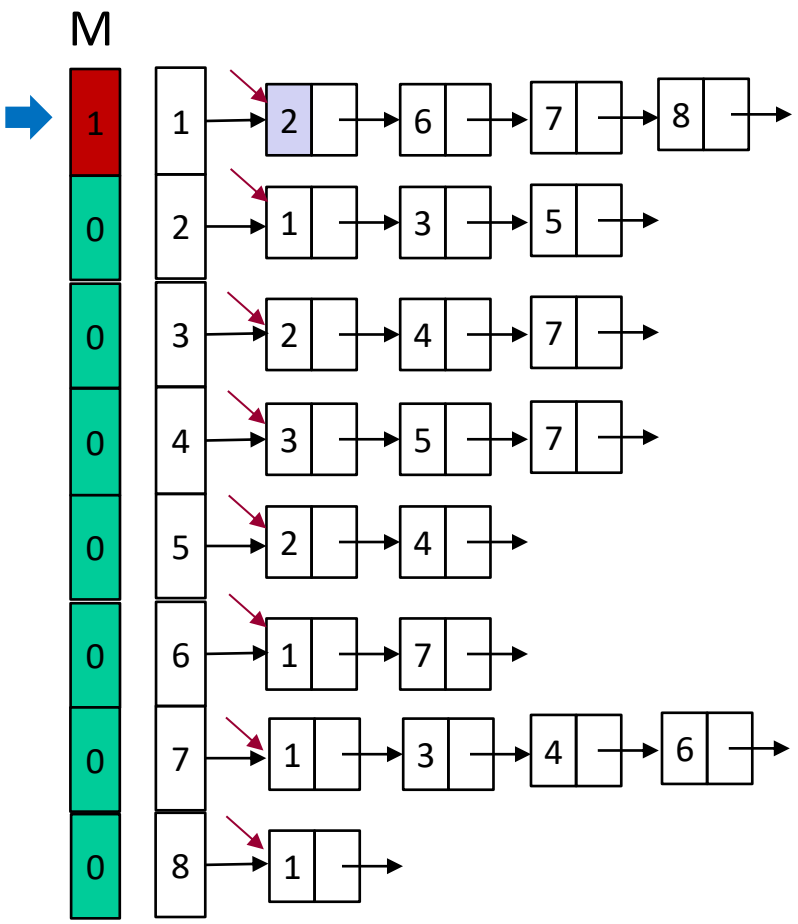
DFS(1)



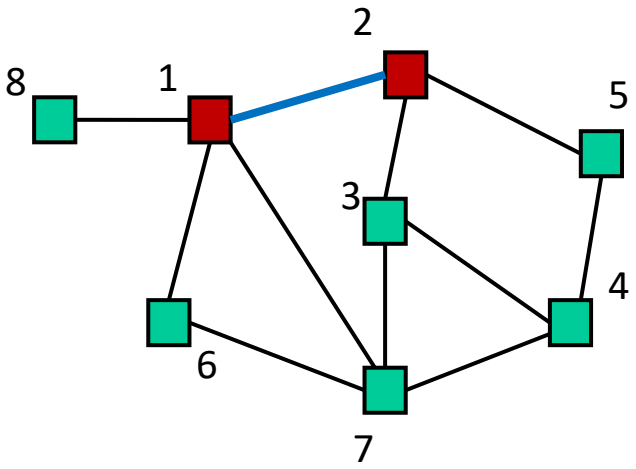
DFS Example – with adj. lists



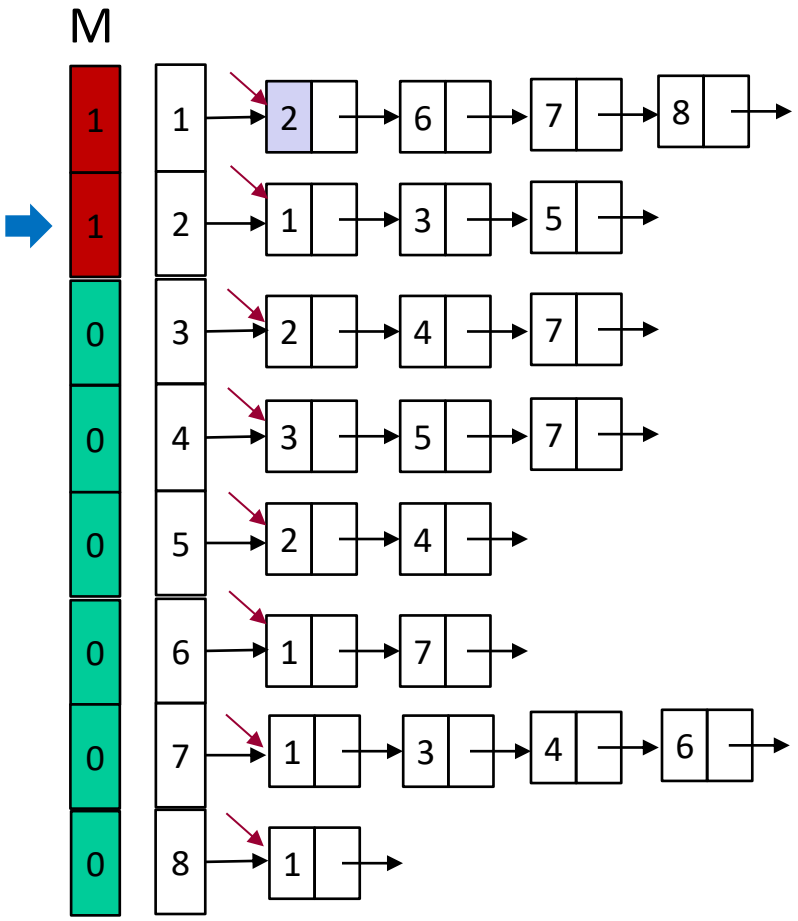
DFS(1)



DFS Example – with adj. lists



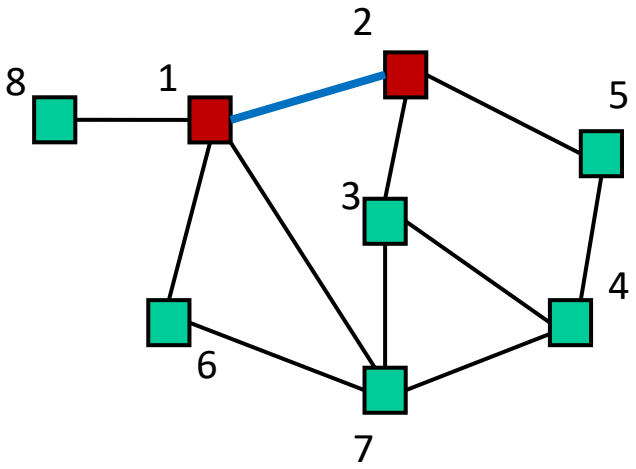
DFS(2)



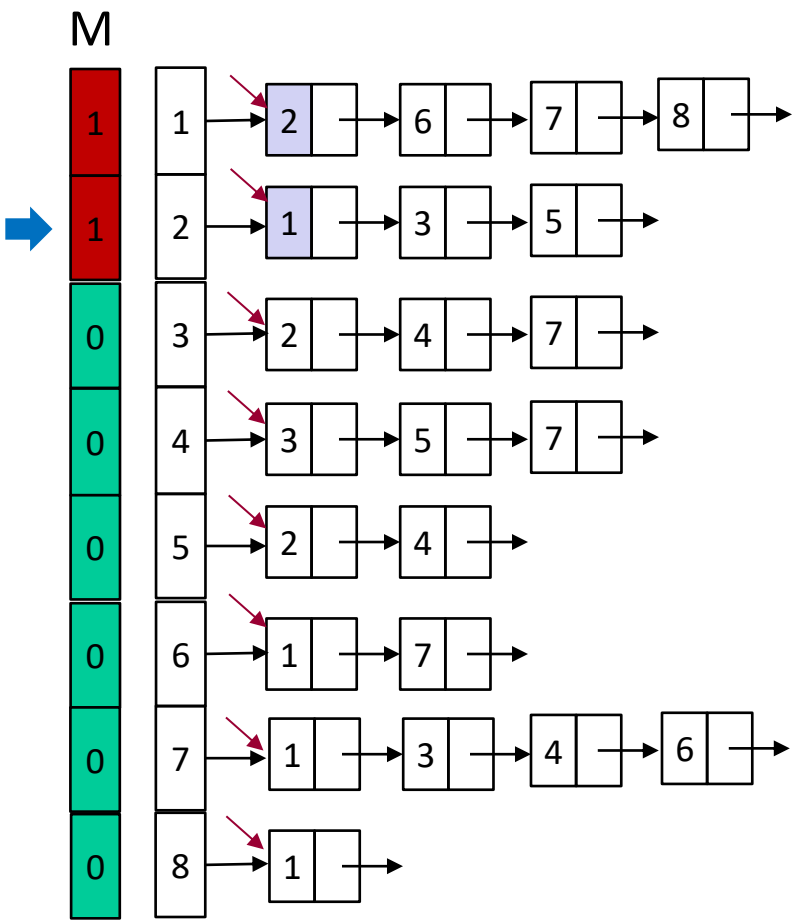
1

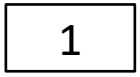
stack

DFS Example – with adj. lists



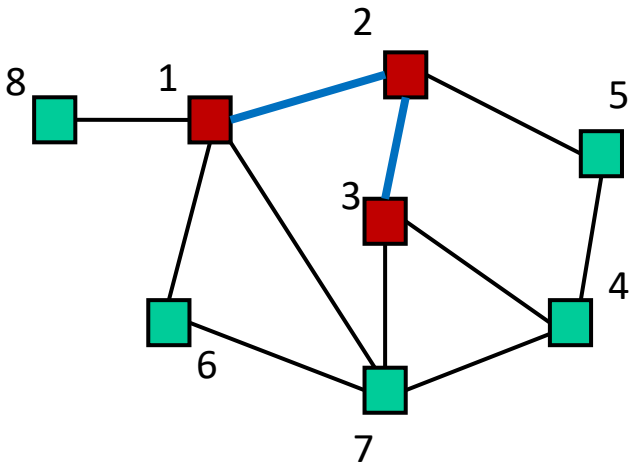
DFS(2)



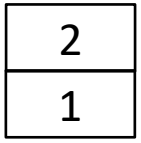


stack

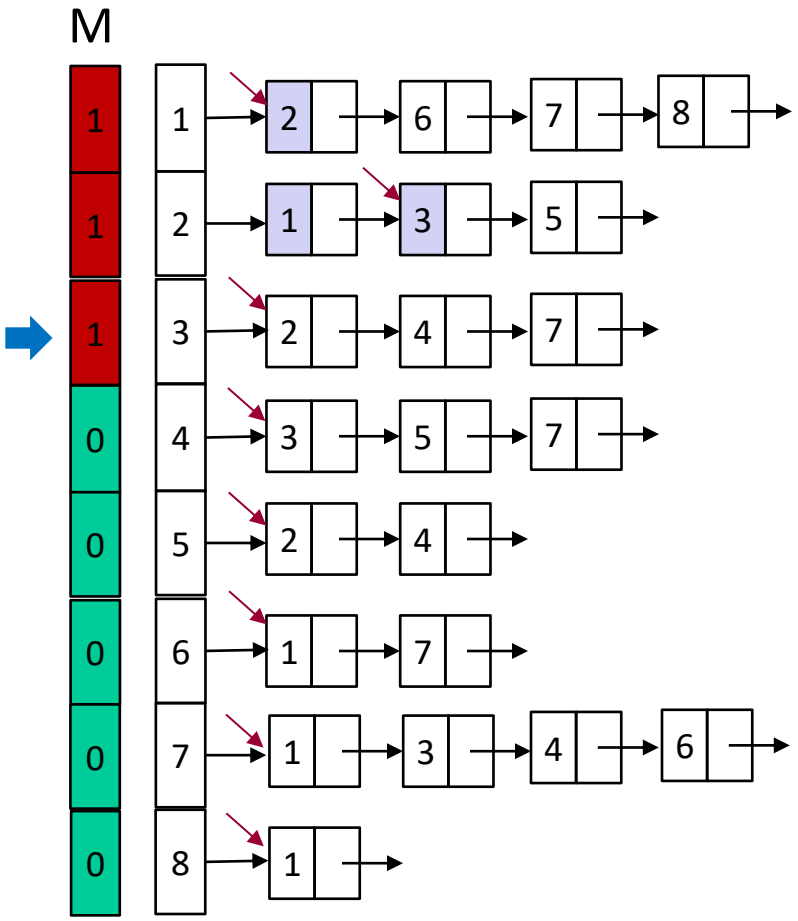
DFS Example – with adj. lists



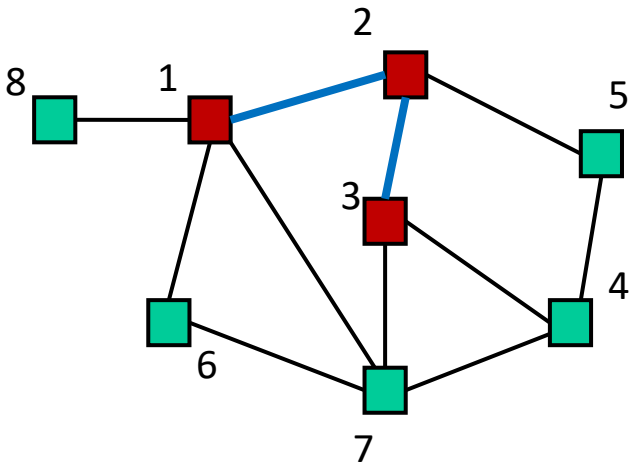
DFS(3)



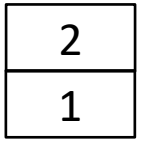
stack



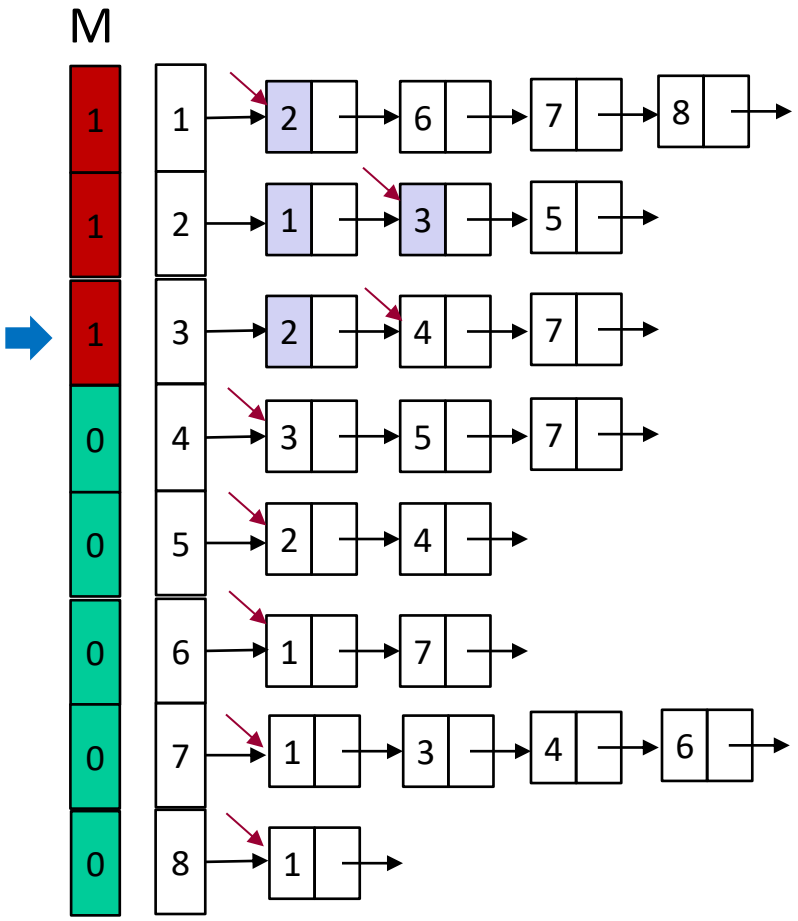
DFS Example – with adj. lists



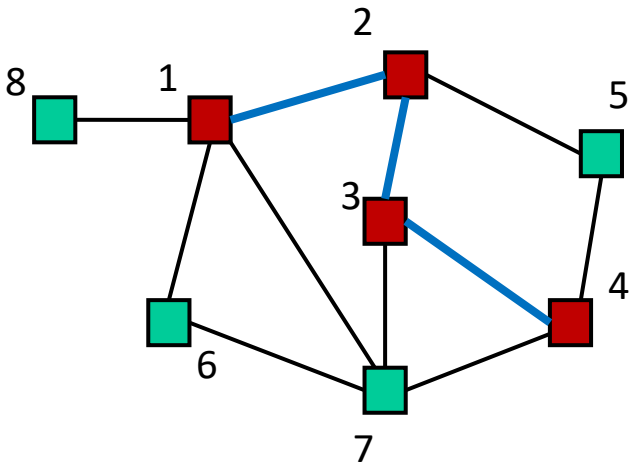
DFS(3)



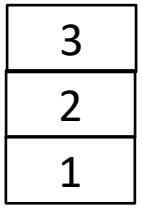
stack



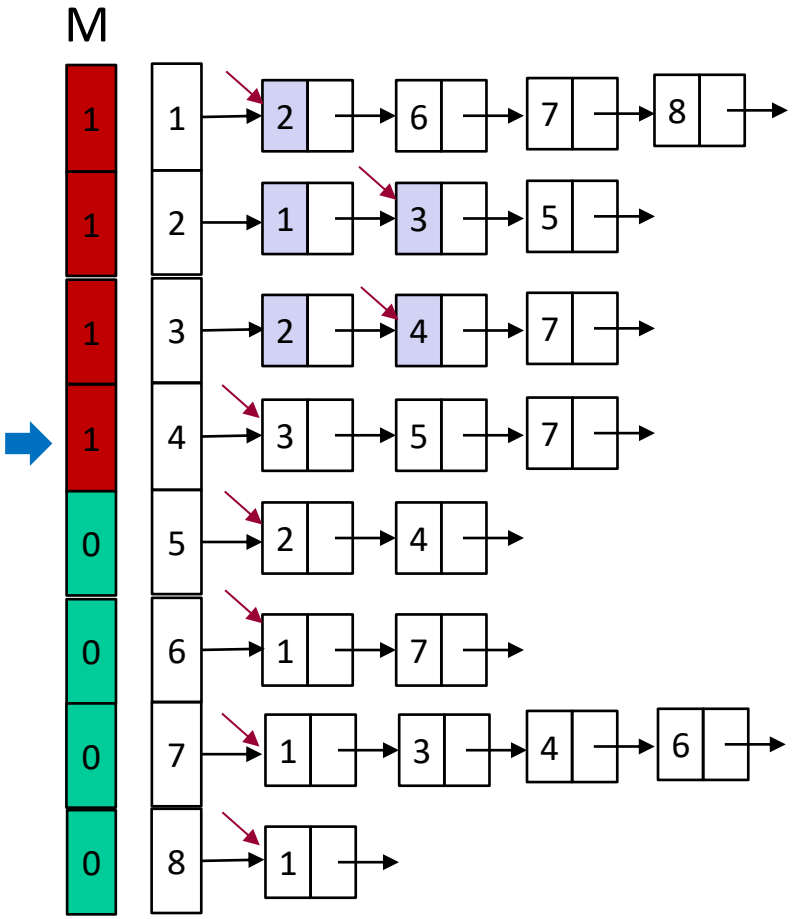
DFS Example – with adj. lists



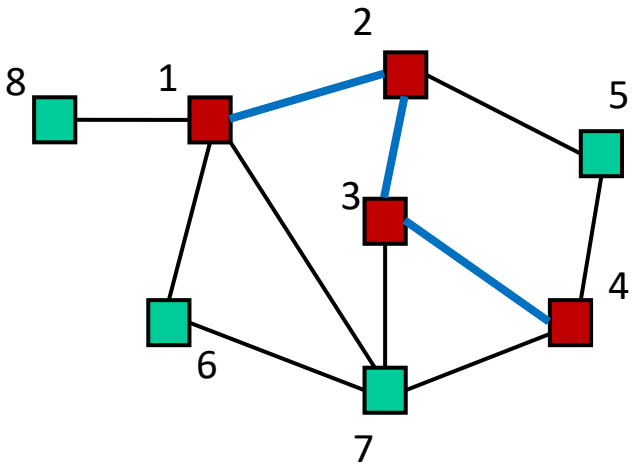
DFS(4)



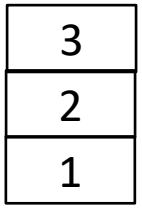
stack



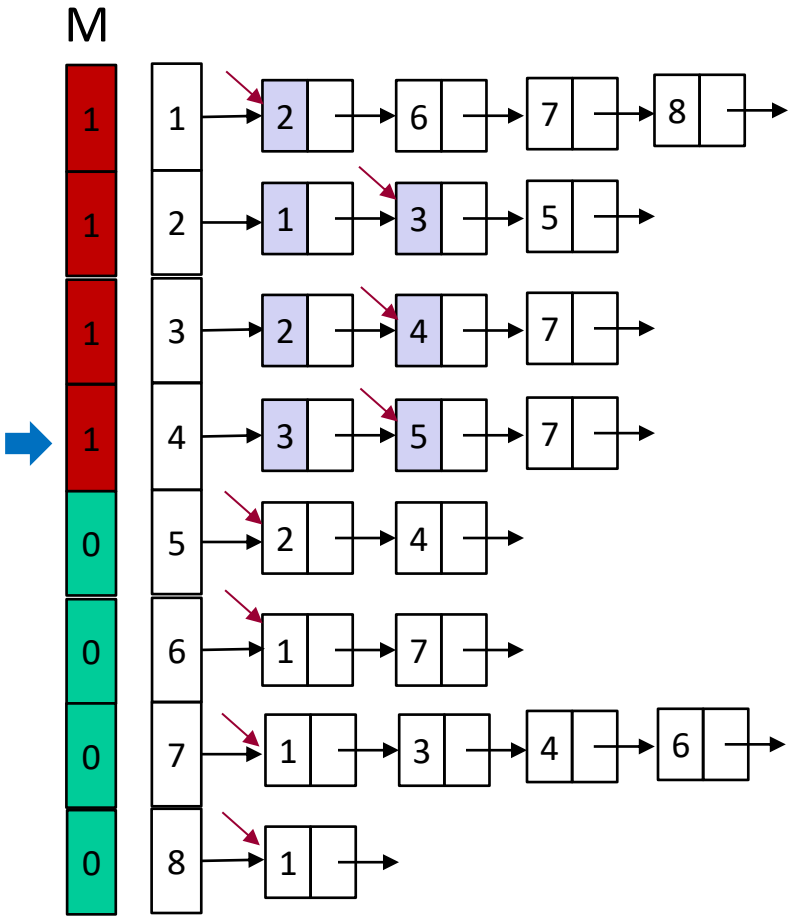
DFS Example – with adj. lists



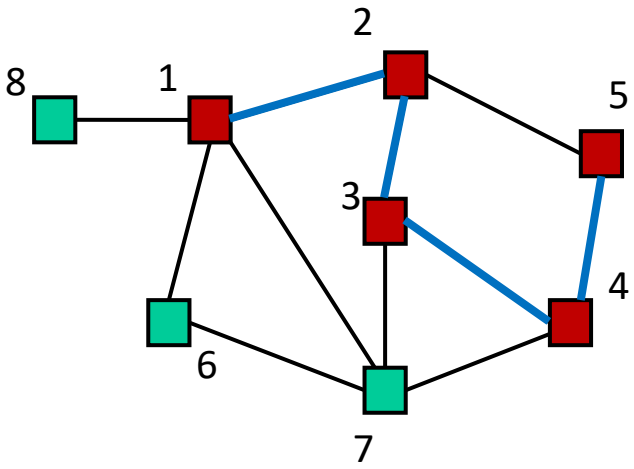
DFS(4)



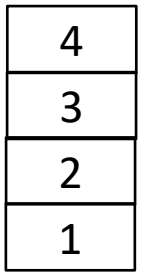
stack



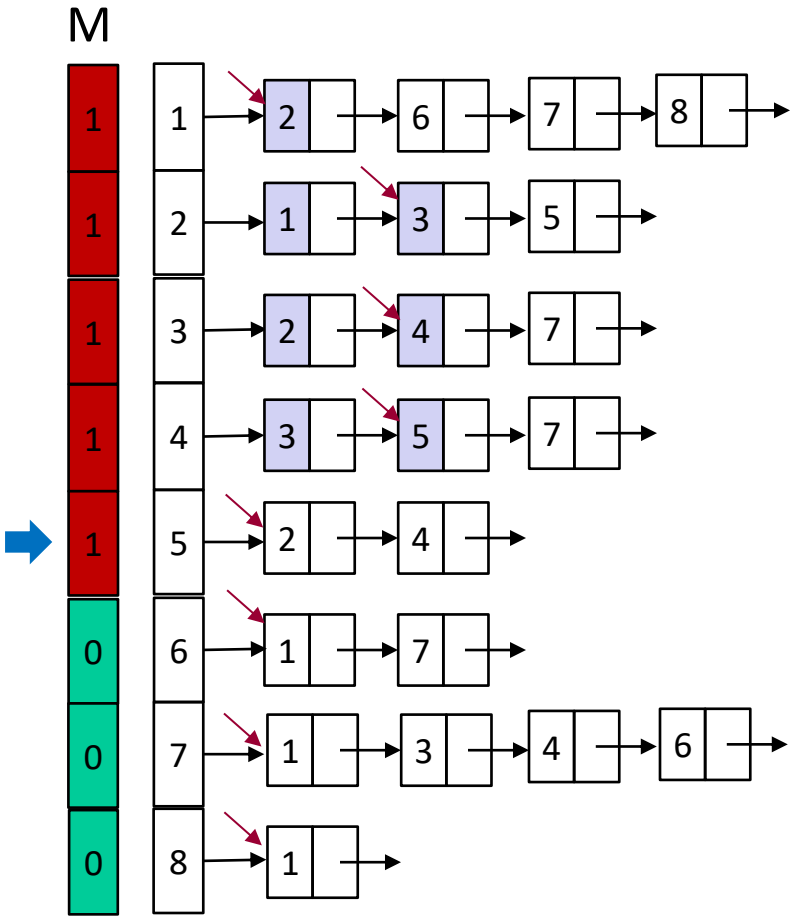
DFS Example – with adj. lists



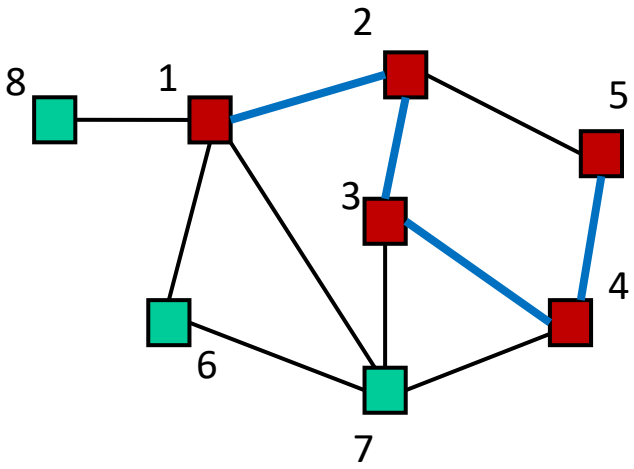
DFS(5)



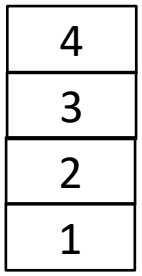
stack



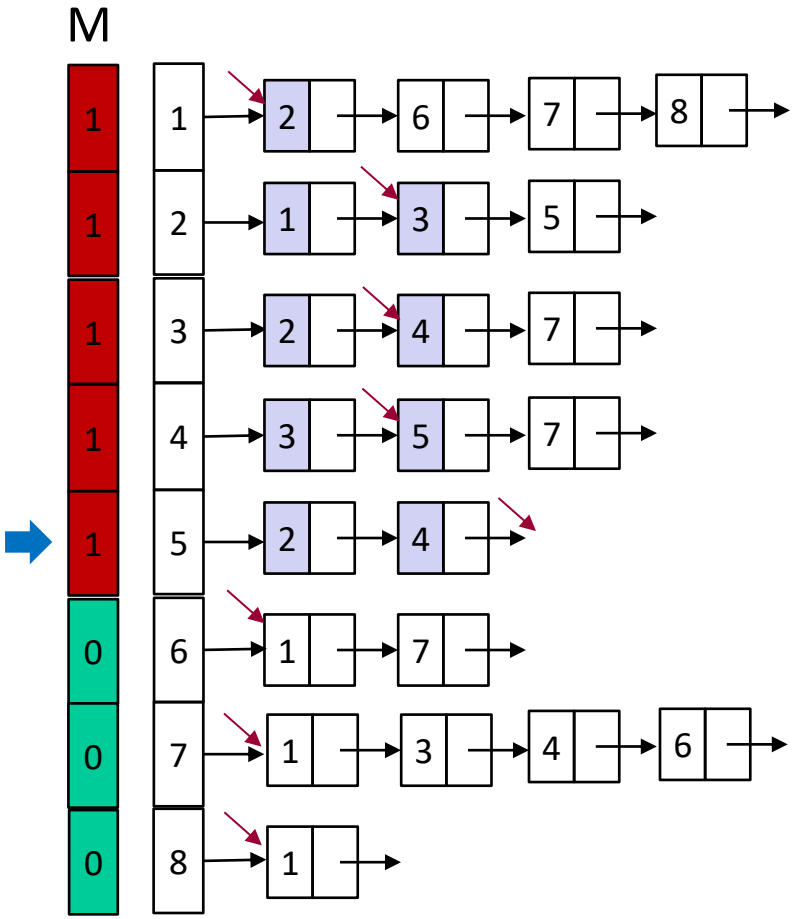
DFS Example – with adj. lists



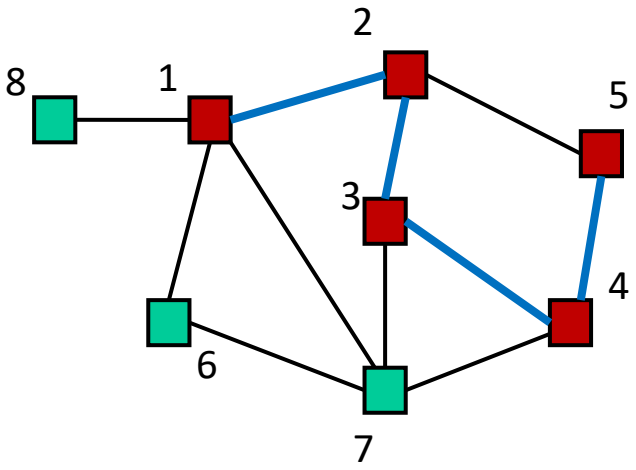
DFS(5)



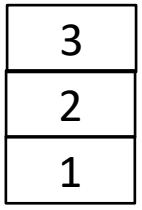
stack



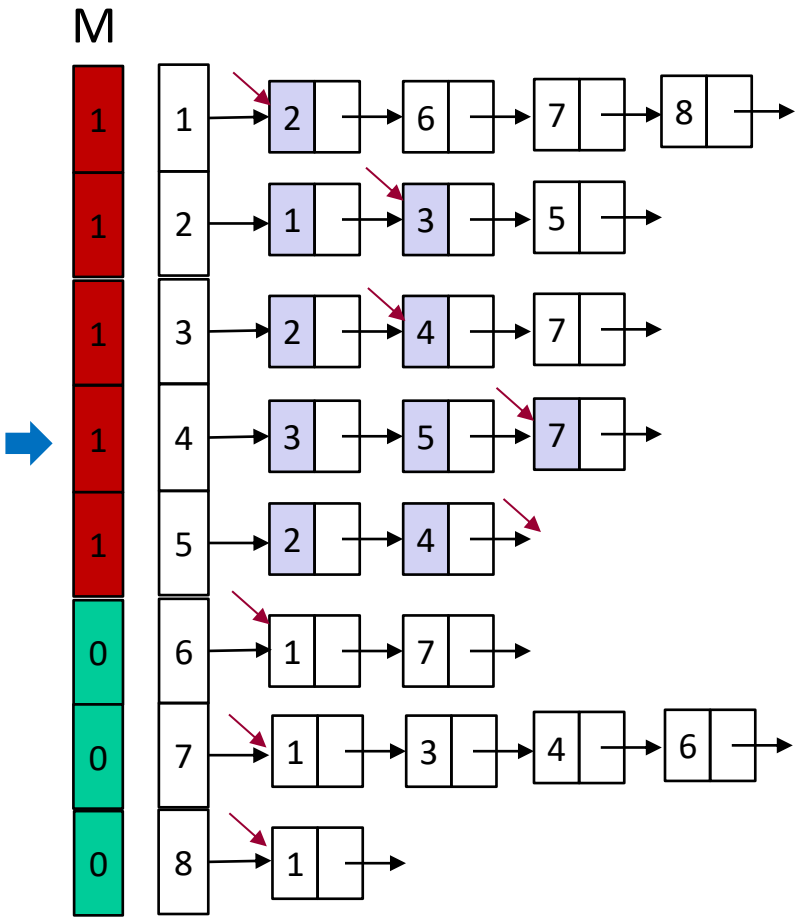
DFS Example – with adj. lists



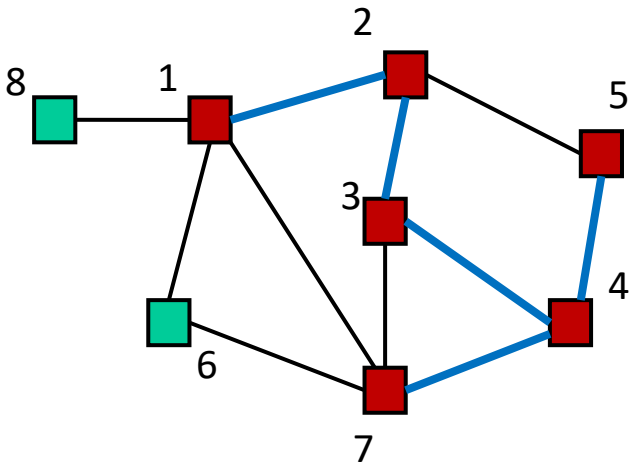
Back to DFS(4)



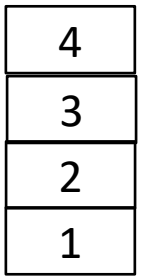
stack



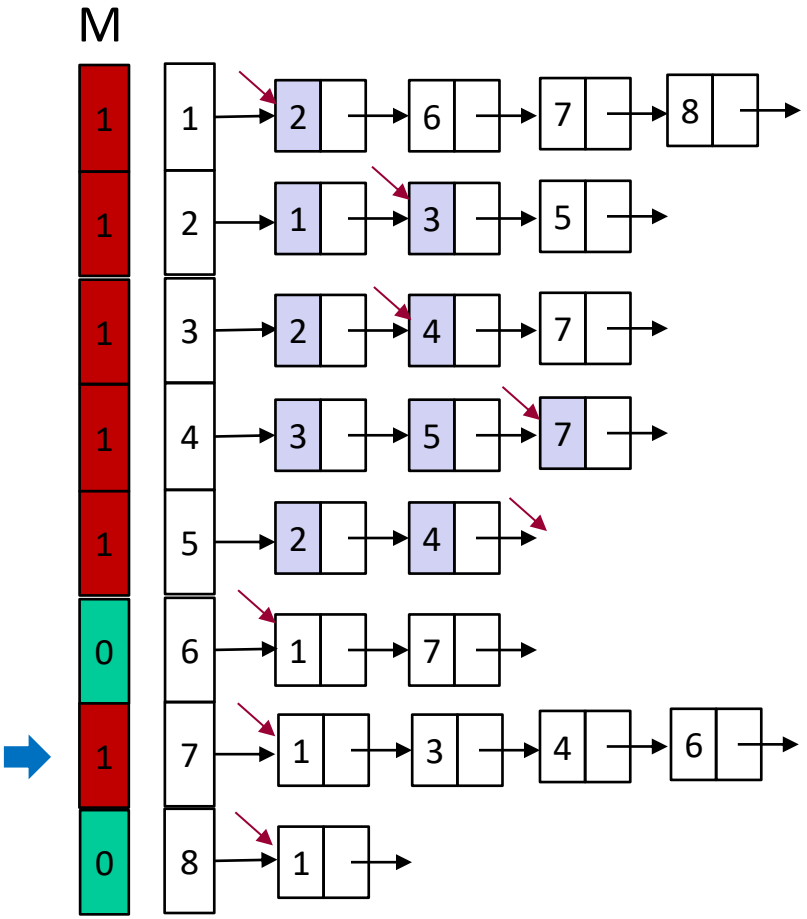
DFS Example – with adj. lists



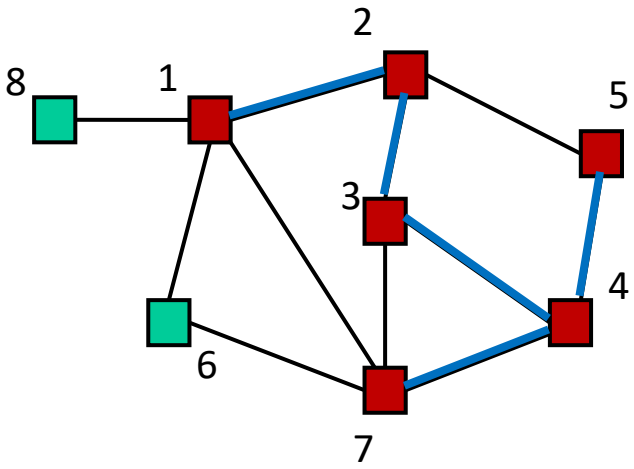
DFS(7)



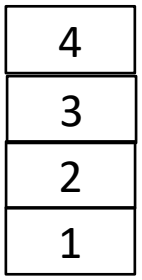
stack



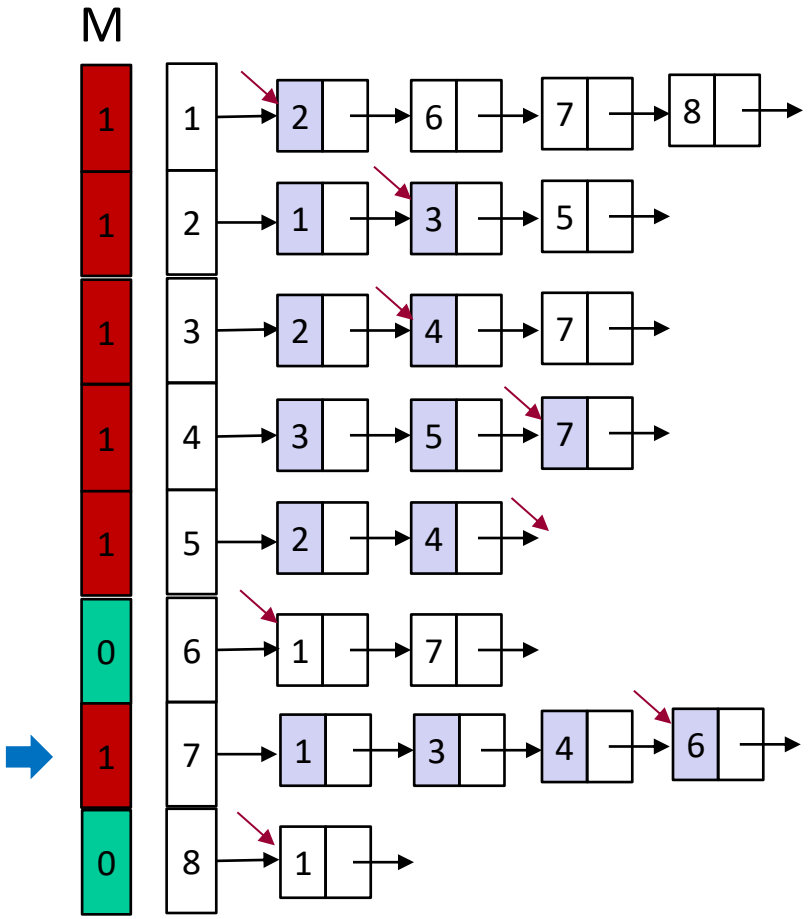
DFS Example – with adj. lists



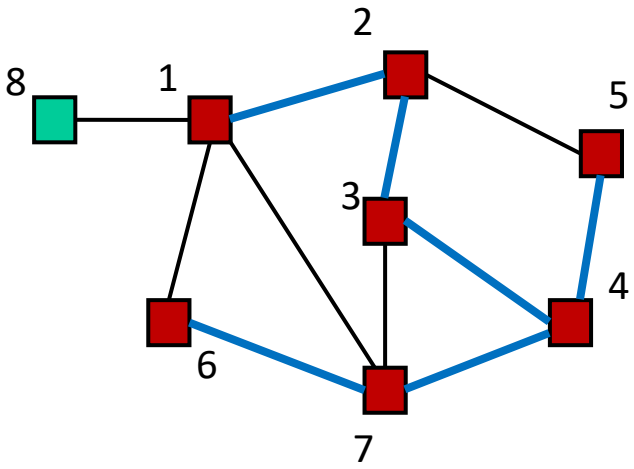
DFS(7)



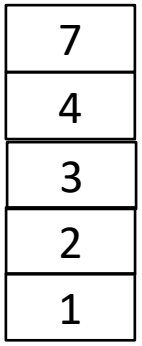
stack



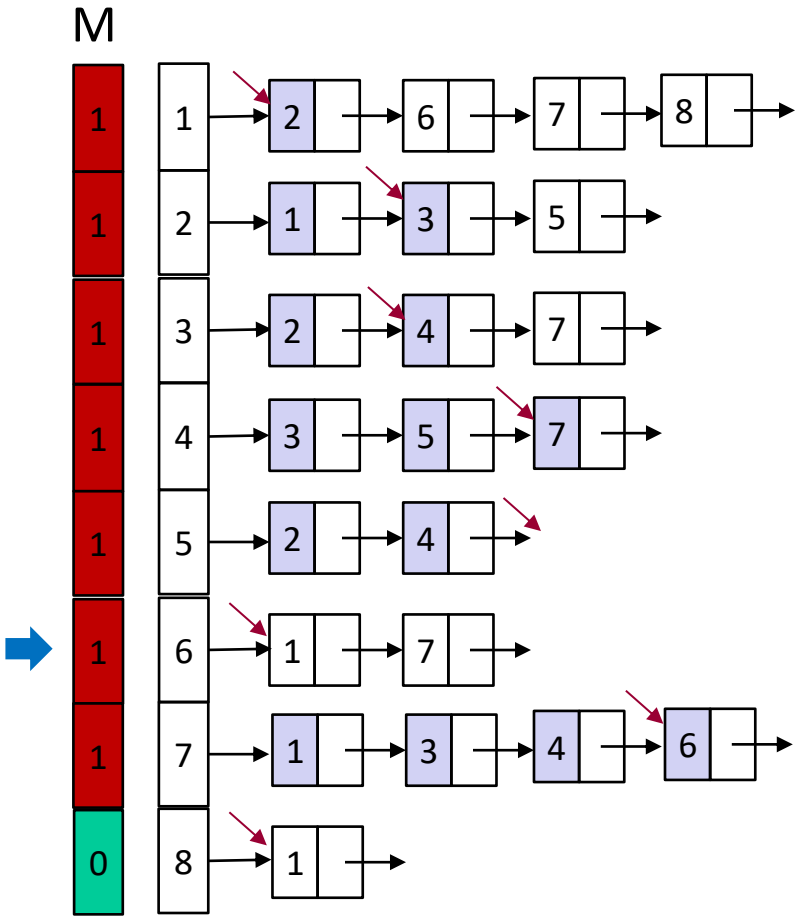
DFS Example – with adj. lists



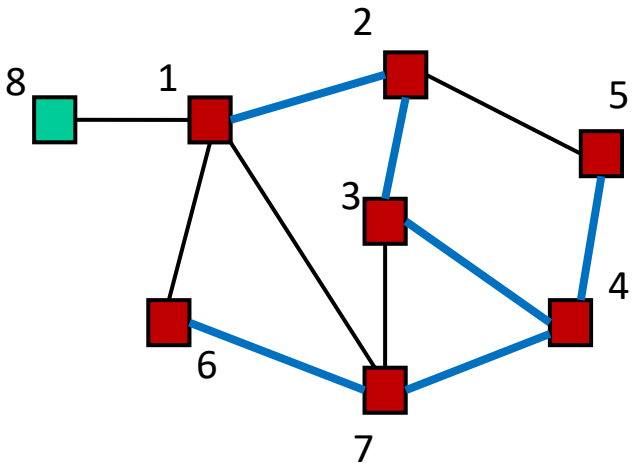
DFS(6)



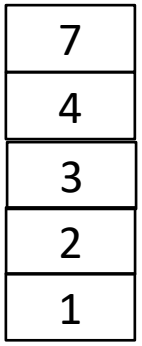
stack



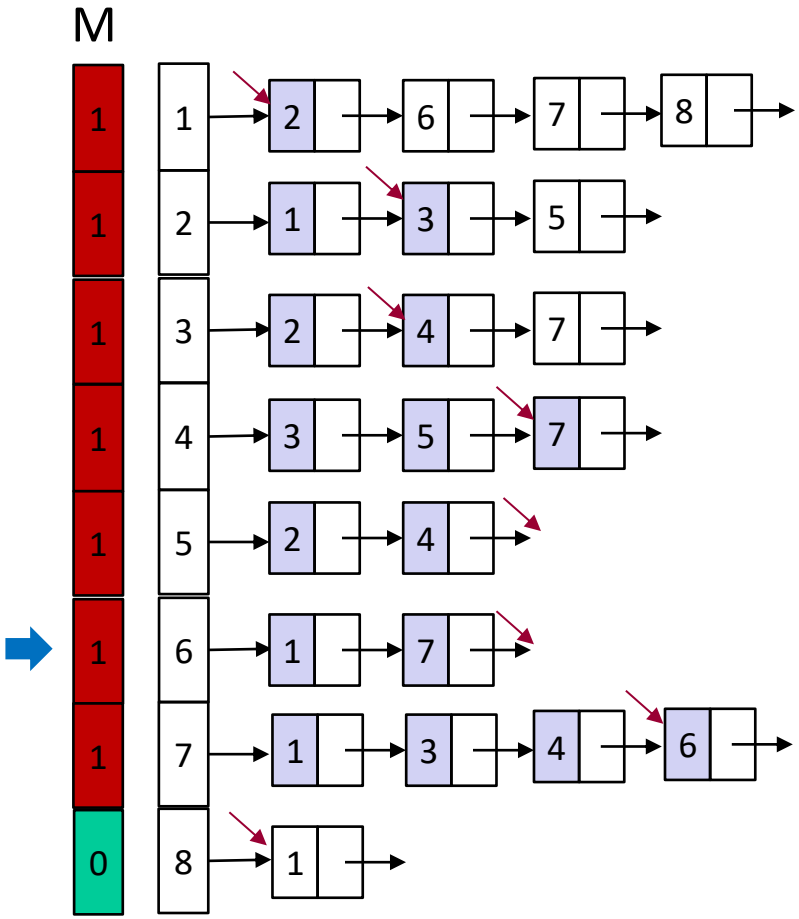
DFS Example – with adj. lists



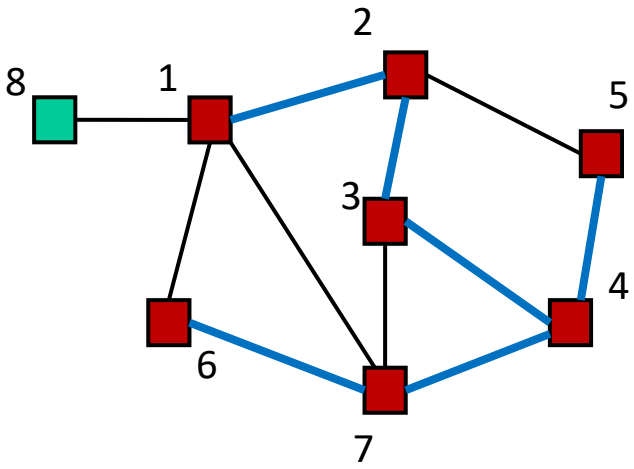
DFS(6)



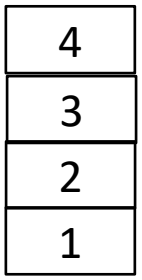
stack



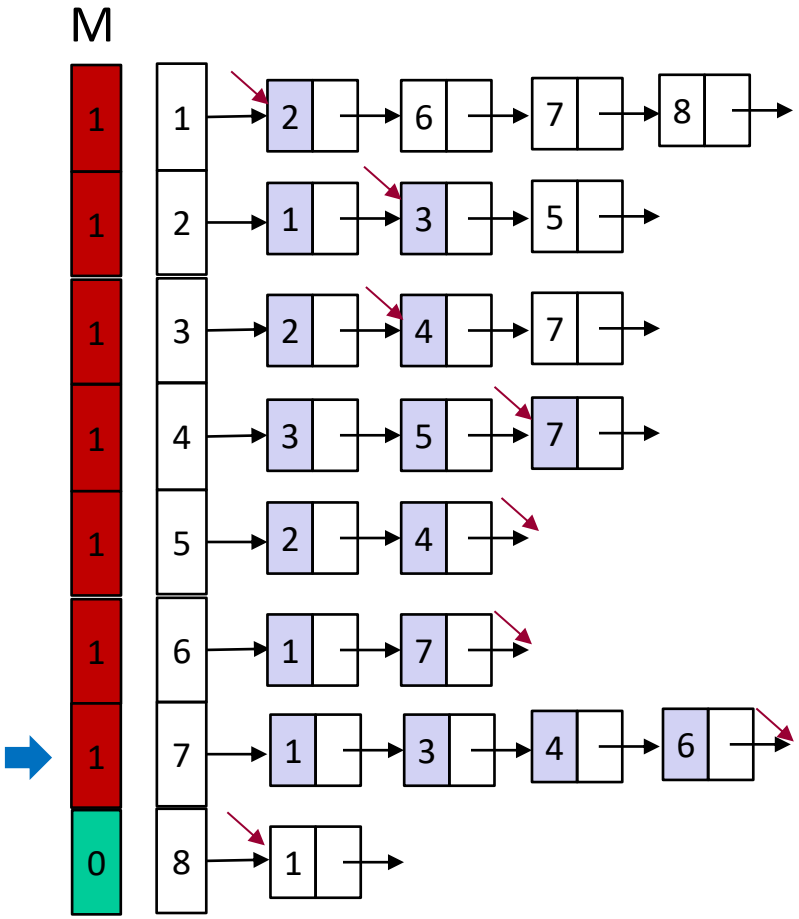
DFS Example – with adj. lists



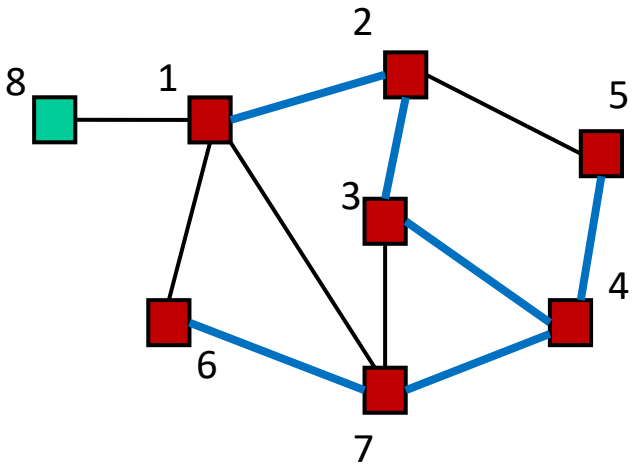
DFS(7)



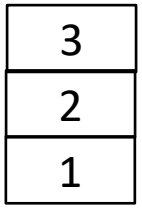
stack



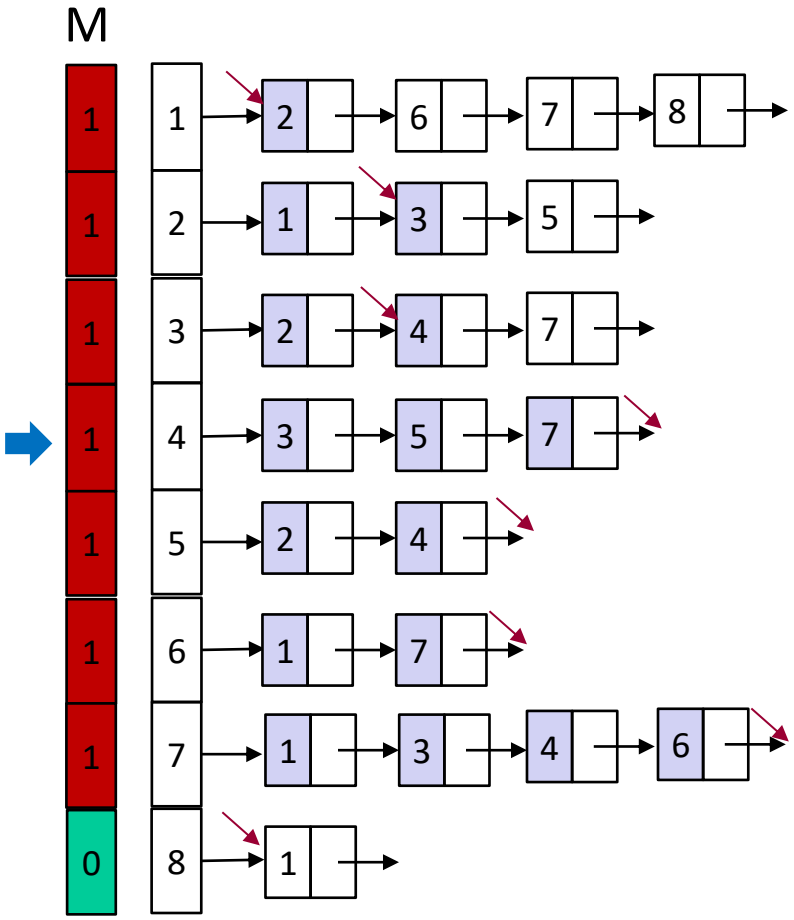
DFS Example – with adj. lists



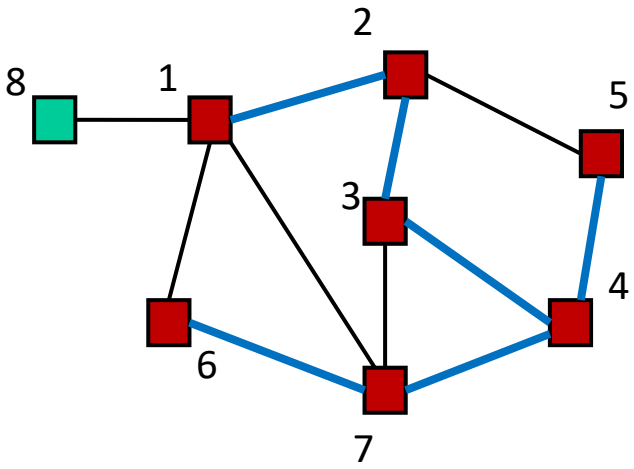
Back to DFS(4)



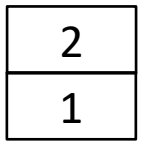
stack



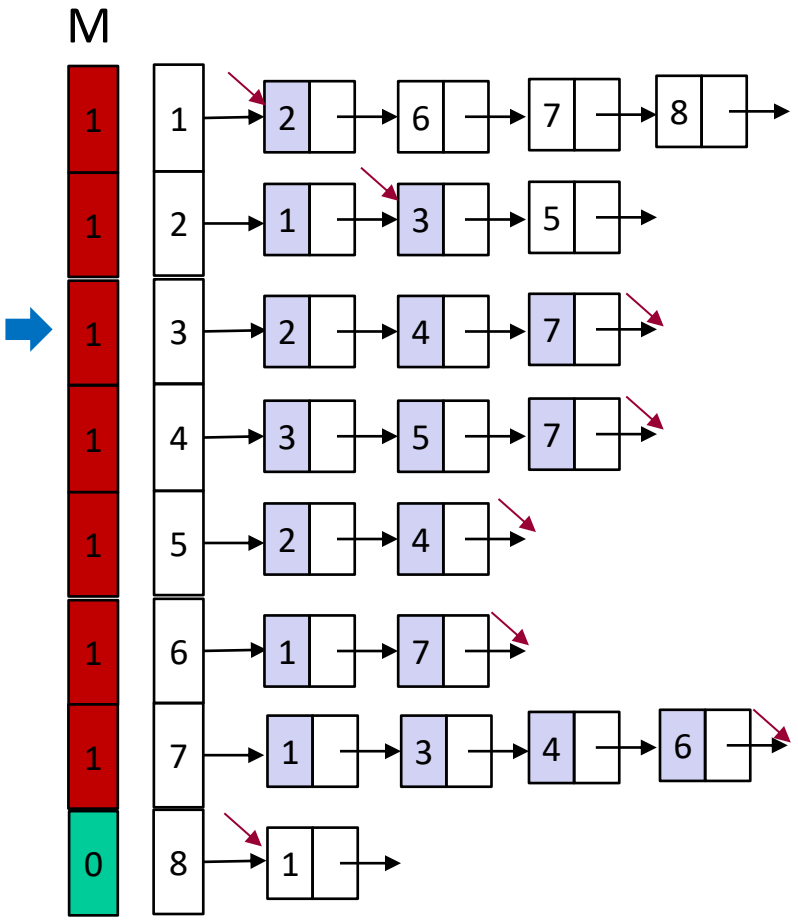
DFS Example – with adj. lists



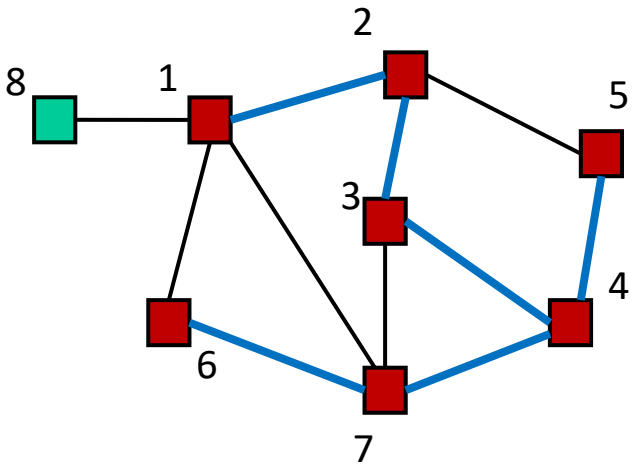
Back to DFS(3)



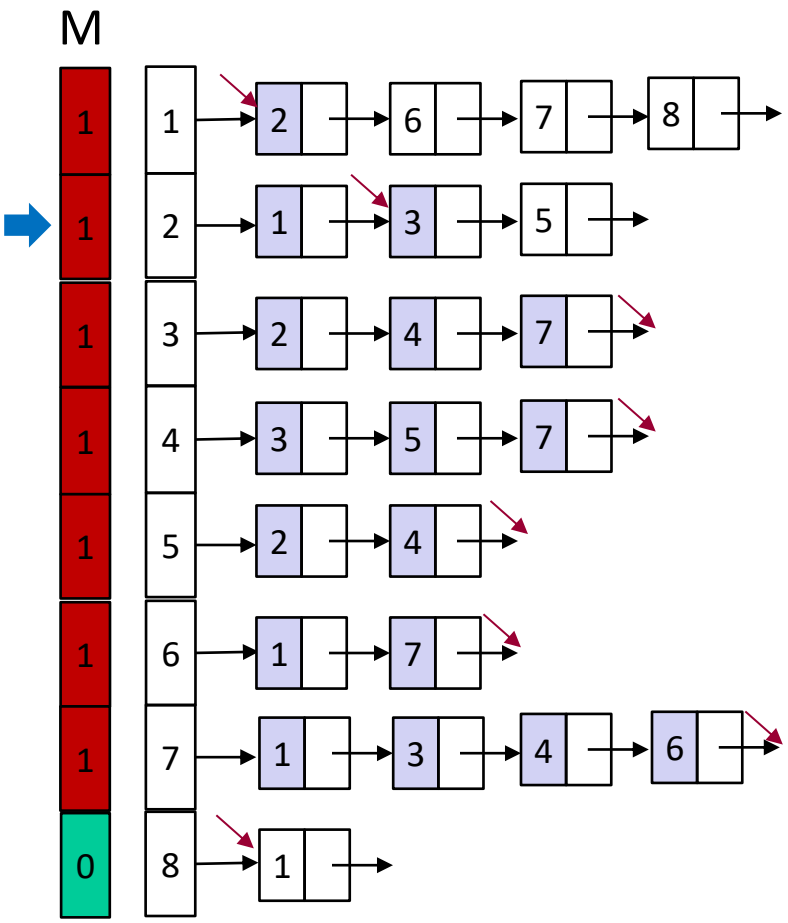
stack

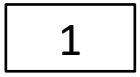


DFS Example – with adj. lists



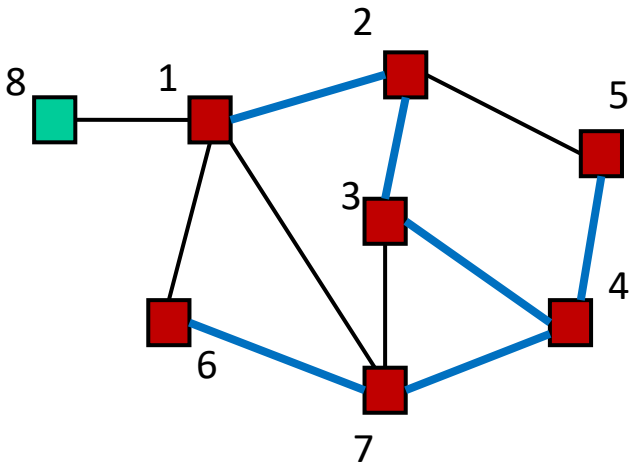
Back to DFS(2)



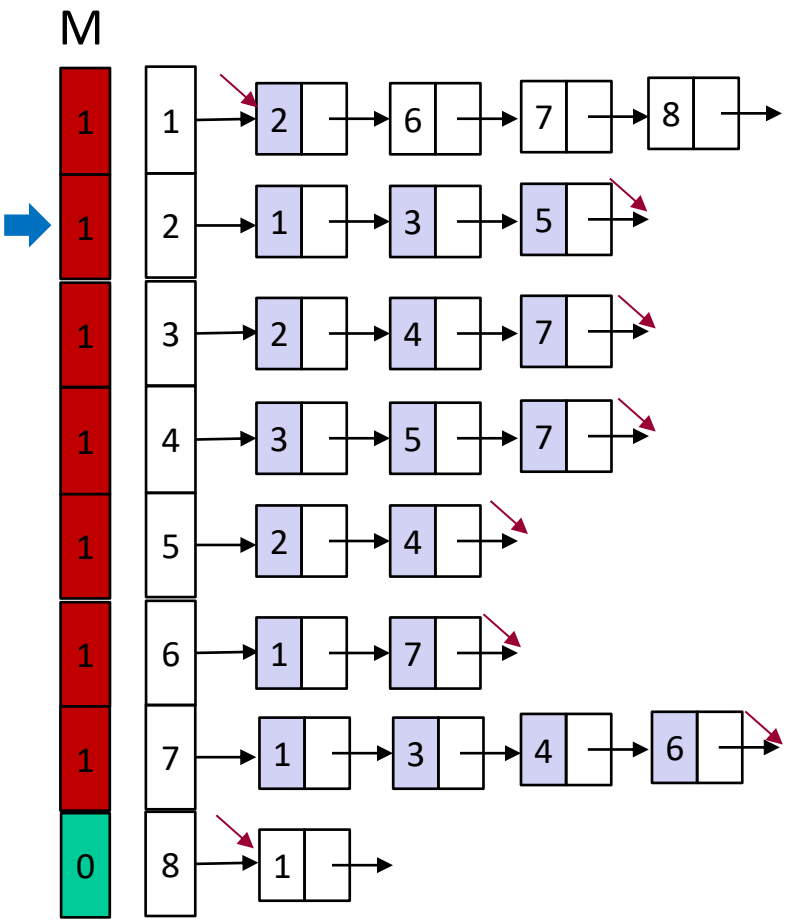


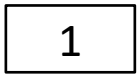
stack

DFS Example – with adj. lists



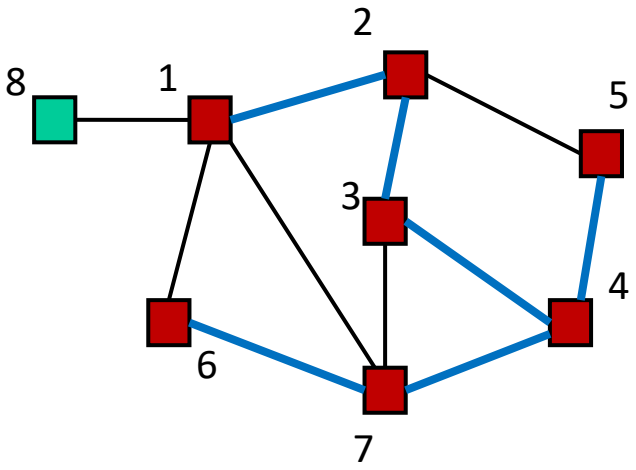
Back to DFS(2)



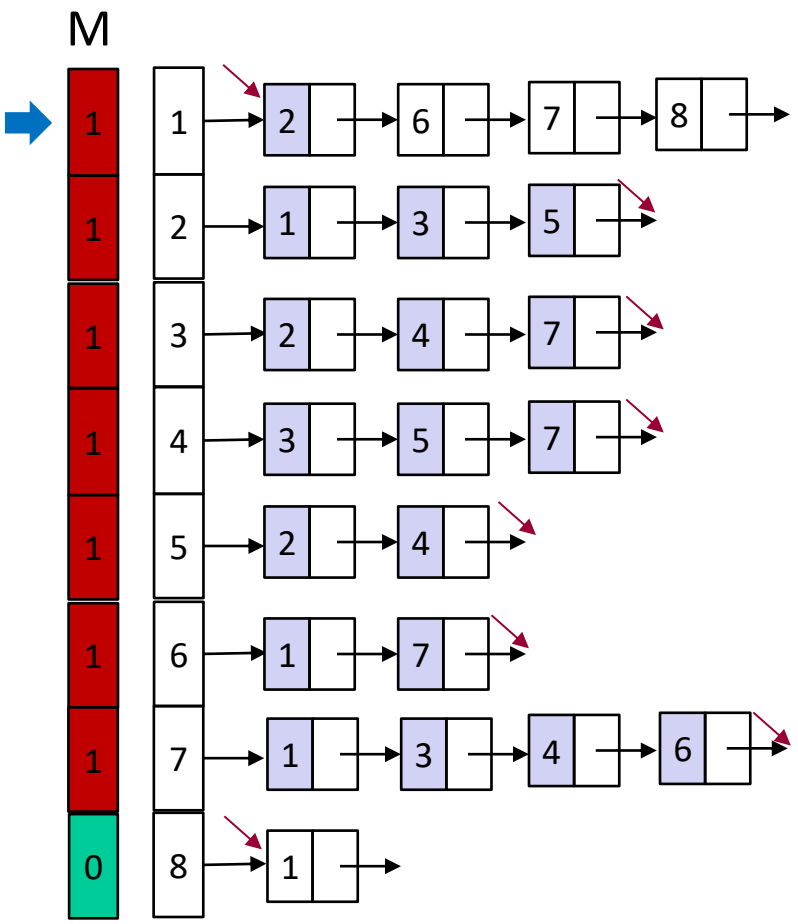


stack

DFS Example – with adj. lists

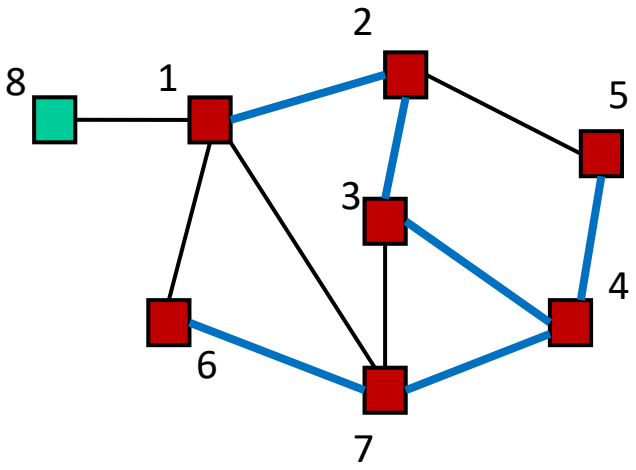


Back to DFS(1)

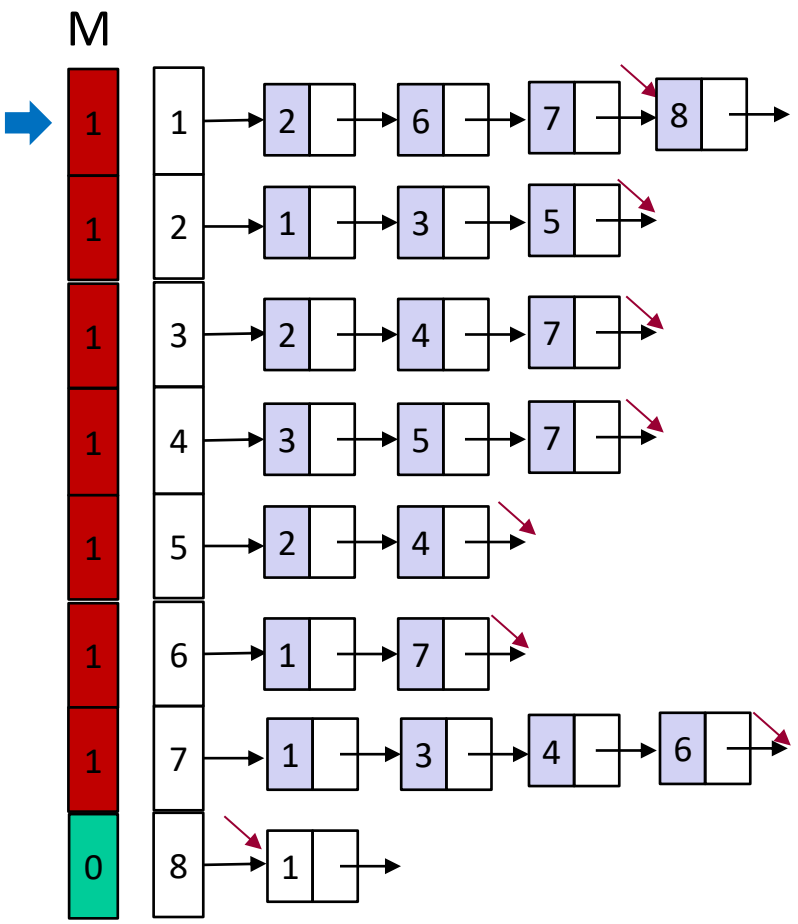


stack

DFS Example – with adj. lists

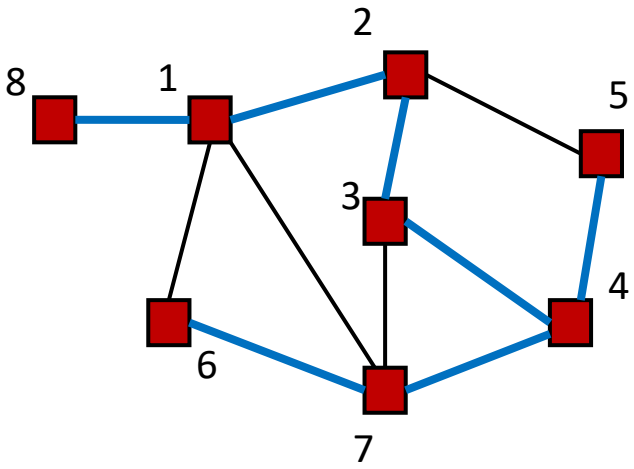


Back to DFS(1)

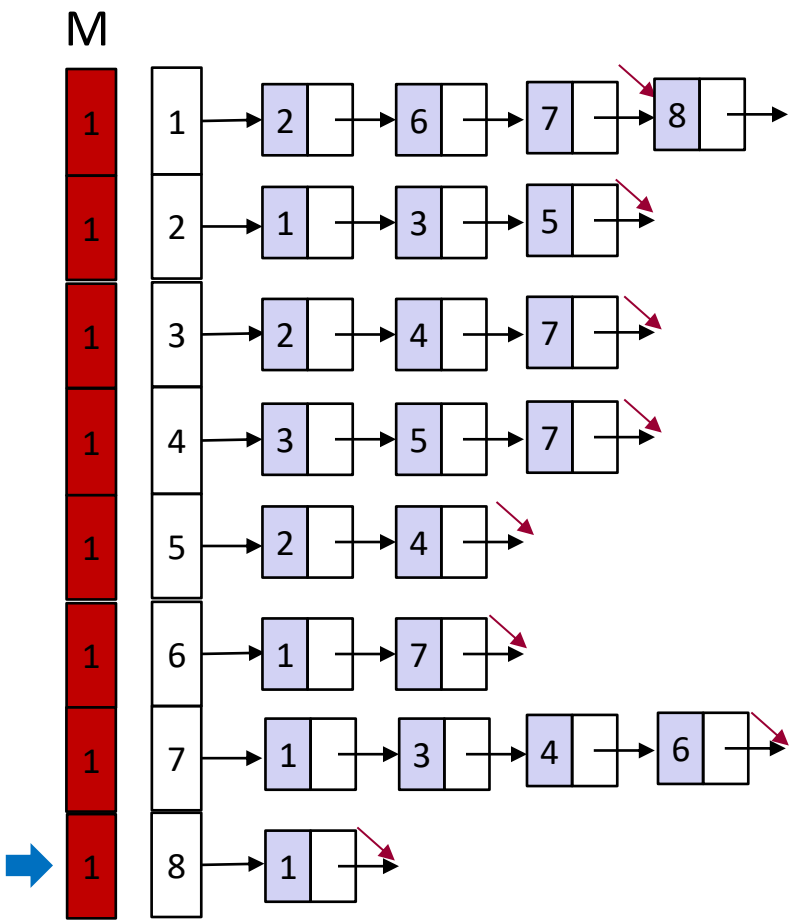


stack

DFS Example – with adj. lists



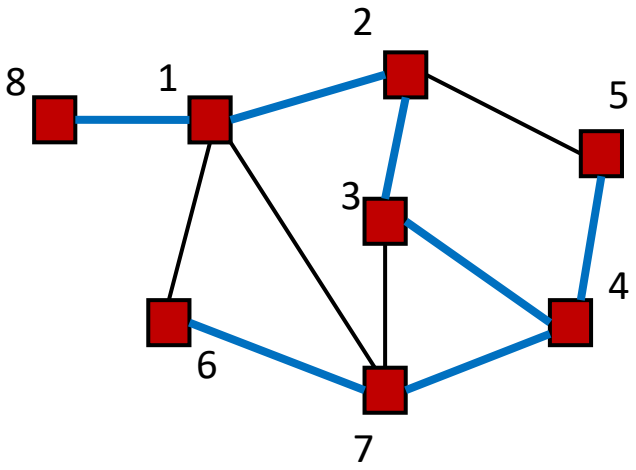
DFS(8)



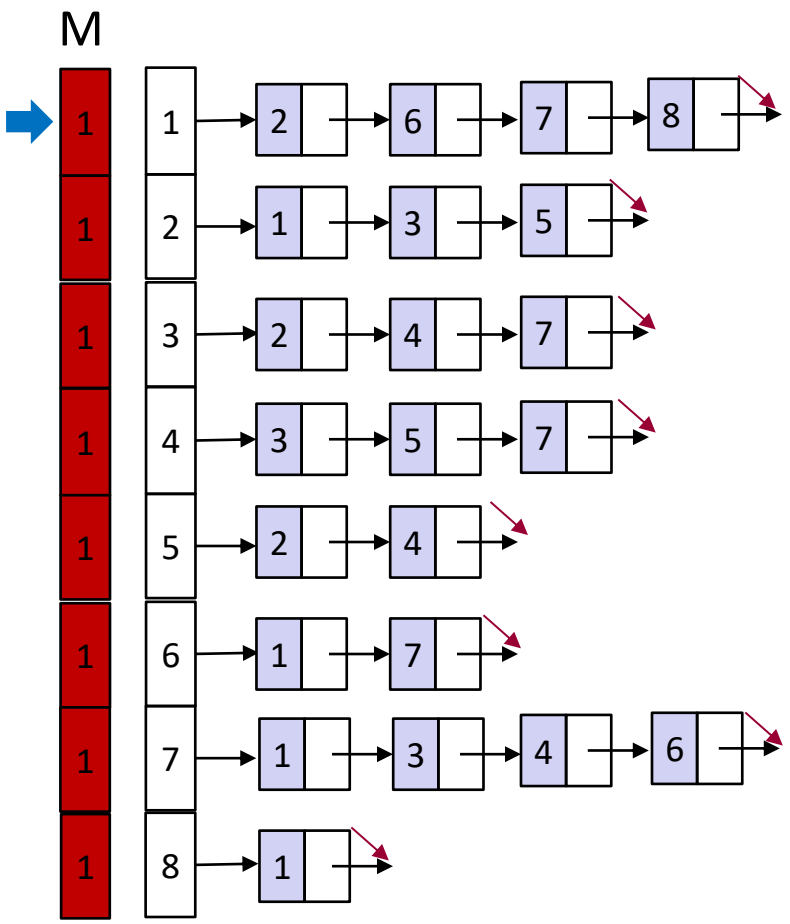
1

stack

DFS Example – with adj. lists

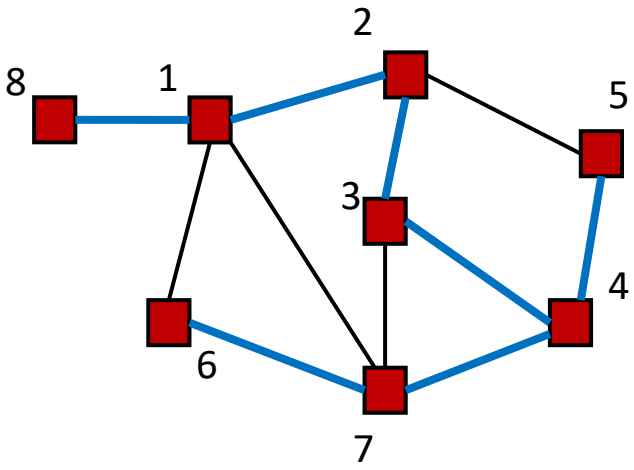


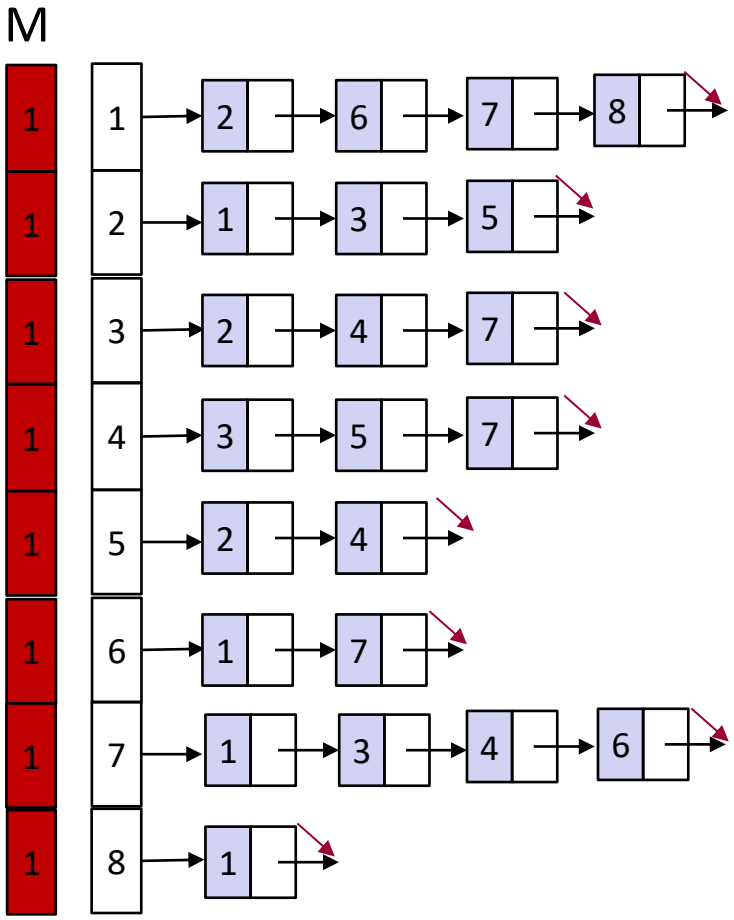
Back to DFS(1)



stack

DFS Example – with adj. lists





Done!



DFS implemented with adj. lists

Time complexity analysis:

Initialization: O(n)

During the algorithm, every edge is considered twice (when each of its endpoint is active). Every time an edge is considered, the algorithm performs O(1) operations ➔ O(m).

Total: O(n+m).

Assuming that the graph is connected, O(n+m)=O(m).

Since $m \geq n-1$

Space complexity analysis:

The graph: O(n+m)

Recursion stack: O(n)

Total: O(n+m)=O(m) assuming G is connected.